Einführung in die fraktale Geometrie am Beispiel der Mandelbrot-Menge

Neufassung 2026

von

Urban Bettag

Facharbeit im LK Mathematik

Integrierte Gesamtschule Ernst Bloch

Ludwigshafen am Rhein

Vorwort

Die vorliegende Facharbeit „Einführung in die fraktale Geometrie am Beispiel der Mandelbrot-Menge“ entstand nach der Lektüre eines Artikels in der deutschsprachigen Ausgabe von „Scientific American“. In dieser Veröffentlichung wurden die Hintergründe eines Objekts der Unterhaltungsmathematik in verständlicher Form dargestellt und faszinierende Computergrafiken gezeigt.

Ziel dieser Facharbeit soll nun sein, das mathematische Gerüst für solche komplexen Strukturen abzuklären. Die Strukturen selbst verbergen tiefe Geheimnisse, deren Erforschung ständig fortschreitet.

Es liegt auf der Hand, dass im Rahmen einer Facharbeit nur in begrenztem Umfang auf das Thema eingegangen werden kann, da es sich hierbei um ein relativ neues Teilgebiet der diskreten Mathematik handelt und die mathematischen Zusammenhänge komplizierter sind, als sie oft dargestellt werden.

Inhaltsverzeichnis

Vorwort **i**

Bildverzeichnis **iii**

1 Einführung in die komplexen Zahlen **1**
1.1 Allgemeines . . . 1
1.2 Definition der komplexen Zahlen . . . 1
1.3 Die vier Grundrechenarten . . . 1
1.3.1 Addition und Subtraktion . . . 1
1.3.2 Multiplikation . . . 1
1.3.3 Division . . . 1
1.4 Grundgesetze der Algebra in $\mathbb{C}$ . . . 2
1.5 Der Absolutbetrag in $\mathbb{C}$ . . . 2
1.6 Geometrische Interpretation . . . 3
1.6.1 Addition und Subtraktion . . . 3
1.6.2 Multiplikation und Division . . . 4
1.7 Anordnung von komplexen Zahlen . . . 4
1.8 Beispiele . . . 5

2 Folgen und Grenzwerte in $\mathbb{C}$ **6**
2.1 Folgen und Konvergenz in $\mathbb{C}$ . . . 6
2.1.1 Definition des Grenzwerts . . . 6
2.2 Grenzwertsätze . . . 6
2.3 Beispiel . . . 7

3 Die Mandelbrot-Menge **8**
3.1 Allgemeines . . . 8
3.2 Mathematische Erfassung der Mandelbrot-Menge . . . 8
3.3 Der „Apfelmännchen“-Algorithmus . . . 9
3.4 Verschiedene Darstellungsarten von Mandelbrot-Mengen . . . 11
3.5 Ähnliche Strukturen innerhalb der Mandelbrot-Menge . . . 11
3.6 Weitere Erkenntnisse über Mandelbrot-Mengen . . . 12

4 Fraktale Dimension **13**
4.1 Die Kochsche Schneeflocke . . . 13
4.2 Fraktale Dimension . . . 13

5 Anhang **14**
5.1 Verschiedene Bilder . . . 14
5.2 Nachwort . . . 26
5.3 Literaturverzeichnis . . . 27
5.4 Pascal-Listing . . . 28
5.5 Python-Nachbearbeitung von 2026 . . . 40

Bildverzeichnis

Komplexe Zahl in kartesischen Koordinaten und Polarkoordinaten 3
Addition und Subtraktion in der Gaußschen Zahlenebene 3
Winkel bei Multiplikation und Division komplexer Zahlen 4
Tafel 1: Beispiele iterierter Funktionssysteme 14
Tafel 2: Mandelbrot-Menge in der komplexen Ebene 15
Tafel 3: Wirkung des Iterationslimits 16
Tafel 4: Vergrößerung eines Randgebiets 17
Tafel 5: Farbige Fraktalabbildungen 18
Tafel 6: Schwarz-Weiß-Neuberechnung 19
Tafel 7: Weitere Miniatur 20
Tafel 8: Zwei Ausschnittsvergrößerungen 21
Tafel 9: Koch-Kurve und Skalierung 22
Tafel 10: Dreidimensionale Darstellung 23
Tafel 11: Differenzierte grafische Darstellung 24
Tafel 12: Ausschnitt mit hoher Rechentiefe 25

Hinweis zur Neufassung 2026

Diese Fassung entstand 2026 auf Grundlage der Facharbeit von 1990. Sie folgt deren Gliederung und Gedankengang, ist aber neu formuliert und deutlich gekürzt: Viele Rechenwege, Beispiele, Abbildungen und einzelne Abschnitte des Originals sind zusammengefasst oder entfallen, mathematisch falsche Aussagen sind berichtigt. Nur ein Teil dieser Änderungen ist in den hervorgehobenen Hinweiskästen und im Änderungsnachweis einzeln gekennzeichnet.

Das Original von 1990 ist mit einer wortgetreuen Transkription und einem Faksimile aller Seiten in einem privaten Archiv erhalten. Die Abbildungen wurden 2026 neu berechnet beziehungsweise neu gezeichnet. Der dazu verwendete Python-Code ist im Anhang getrennt vom originalen Pascal-Code von 1990 dokumentiert.

1 Einführung in die komplexen Zahlen

1.1 Allgemeines

In der Algebra trifft man beim Lösen von Gleichungen, zum Beispiel bei der Nullstellenberechnung, öfter auf das Problem, dass eine Gleichung keine reelle Lösung besitzt. Innerhalb der reellen Zahlen kann keine Wurzel aus einer negativen Zahl gezogen werden. Diese Einschränkung lässt sich überwinden, indem man den Zahlenbereich erweitert und eine neue Zahl i mit der Eigenschaft $i^2 = -1$ einführt.

1.2 Definition der komplexen Zahlen

Eine komplexe Zahl besteht aus einem reellen und einem imaginären Anteil. Die Menge der komplexen Zahlen wird definiert durch

$$\mathbb{C} = \{\, a + bi \mid a,b \in \mathbb{R} \,\}, \qquad i^2 = -1.$$

Für $z = a + bi$ heißt $a = \mathrm{Re}(z)$ Realteil und $b = \mathrm{Im}(z)$ Imaginärteil. Die Bezeichnung $i = \sqrt{-1}$ ist nur als anschauliche Kurzschreibweise zu verstehen; die eigentliche Definition lautet $i^2 = -1$.

1.3 Die vier Grundrechenarten

Seien $z_1 = a + bi$ und $z_2 = c + di$ mit $a,b,c,d \in \mathbb{R}$.

1.3.1 Addition und Subtraktion

$$z_1 + z_2 = (a + c) + i(b + d), \qquad z_1 - z_2 = (a - c) + i(b - d).$$

Komplexe Zahlen werden addiert beziehungsweise subtrahiert, indem man ihre Realteile und ihre Imaginärteile getrennt addiert beziehungsweise subtrahiert.

1.3.2 Multiplikation

$$z_1 z_2 = (a + bi)(c + di) = (ac - bd) + i(ad + bc).$$

Dabei wird wie bei reellen Termen ausmultipliziert und anschließend $i^2 = -1$ eingesetzt. Für die konjugiert komplexe Zahl $\overline{z} = a - bi$ gilt insbesondere

$$z\overline{z} = (a + bi)(a - bi) = a^2 + b^2.$$

1.3.3 Division

Für $z_2 \neq 0$, also $c^2 + d^2 \neq 0$, wird der Bruch mit der konjugiert komplexen Zahl des Nenners erweitert:

$$\frac{z_1}{z_2} = \frac{a + bi}{c + di} = \frac{(a + bi)(c - di)}{(c + di)(c - di)} = \frac{ac + bd}{c^2 + d^2} + i\,\frac{bc - ad}{c^2 + d^2}.$$

Beide Bestandteile des Ergebnisses sind reelle Zahlen.

1.4 Grundgesetze der Algebra in $\mathbb{C}$

Die aus den reellen Zahlen bekannten Rechengesetze gelten auch für komplexe Zahlen. Für $z_1,z_2,z_3 \in \mathbb{C}$ gelten das Kommutativgesetz

$$z_1 + z_2 = z_2 + z_1, \qquad z_1 z_2 = z_2 z_1,$$

das Assoziativgesetz

$$(z_1 + z_2) + z_3 = z_1 + (z_2 + z_3), \qquad (z_1 z_2) z_3 = z_1 (z_2 z_3),$$

und das Distributivgesetz

$$z_1 (z_2 + z_3) = z_1 z_2 + z_1 z_3.$$

Die neutralen Elemente sind $0 = 0 + 0i$ für die Addition und $1 = 1 + 0i$ für die Multiplikation. Zu $z = a + bi$ gehört das additive Inverse $-z = -a - bi$. Für $z \neq 0$ existiert außerdem das multiplikative Inverse

$$z^{-1} = \frac{\overline{z}}{z\overline{z}} = \frac{a - bi}{a^2 + b^2}.$$

Damit sind die komplexen Zahlen mit Addition und Multiplikation ein Körper.

> **Korrekturhinweis** Im Original wurden einzelne Umformungen ausführlich vorgerechnet, die neutralen Elemente und das multiplikative Inverse aber nicht vollständig angegeben. Diese von der Lehrkraft markierte Lücke ist hier geschlossen.

1.5 Der Absolutbetrag in $\mathbb{C}$

Der Betrag einer komplexen Zahl $z = a + bi$ ist definiert durch

$$|z| = |a + bi| = \sqrt{a^2 + b^2}.$$

Es gelten

$$|z| \geq 0, \qquad |z| = |-z|, \qquad |z_1 z_2| = |z_1|\,|z_2|$$

und für $z_2 \neq 0$

$$\left|\frac{z_1}{z_2}\right| = \frac{|z_1|}{|z_2|}.$$

Außerdem gilt die Dreiecksungleichung

$$|z_1 + z_2| \leq |z_1| + |z_2|.$$

Zum Nachweis kann man beide nichtnegativen Seiten quadrieren. Mit $z_1 = a + bi$ und $z_2 = c + di$ genügt dann

$$ac + bd \leq \sqrt{a^2 + b^2}\sqrt{c^2 + d^2},$$

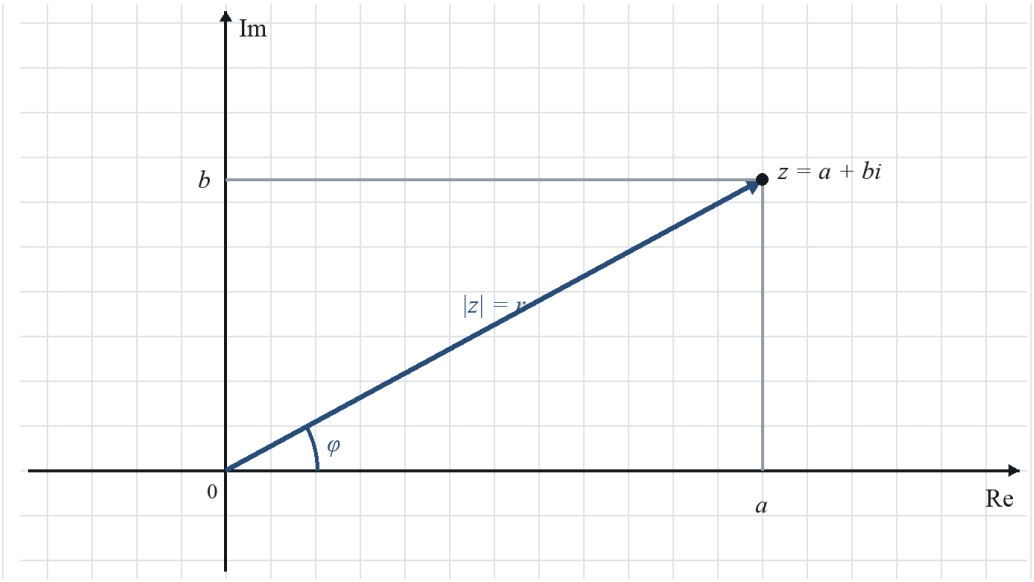
also die Cauchy–Schwarz-Ungleichung in $\mathbb{R}^2$. Äquivalent endet die im Original verwendete Umformung bei der wahren Aussage

$$0 \leq (ad - bc)^2.$$

In der Gaußschen Zahlenebene entspricht $z = a + bi$ dem Punkt (a,b). Der Betrag $|z|$ ist sein Abstand vom Ursprung, und $|z_1 - z_2|$ ist der Abstand der zugehörigen Punkte. Die Bedingungen $|z| = 1$, $|z| < 1$ und $|z| > 1$ beschreiben den Einheitskreis, sein Inneres und sein Äußeres.

1.6 Geometrische Interpretation

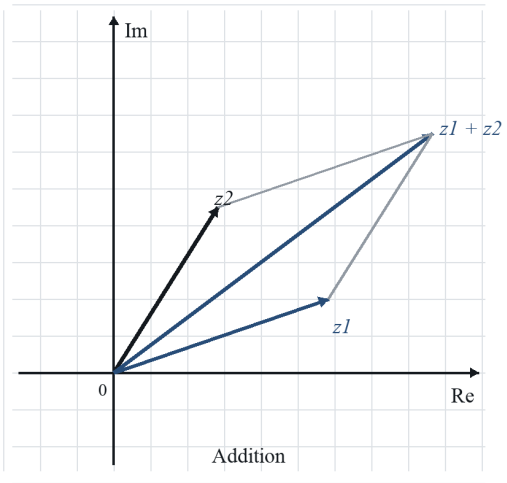
Auf der waagerechten Achse der Gaußschen Zahlenebene wird der Realteil, auf der senkrechten Achse der Imaginärteil abgetragen. Addition und Subtraktion lassen sich dadurch als Vektoraddition beziehungsweise Vektorsubtraktion darstellen.

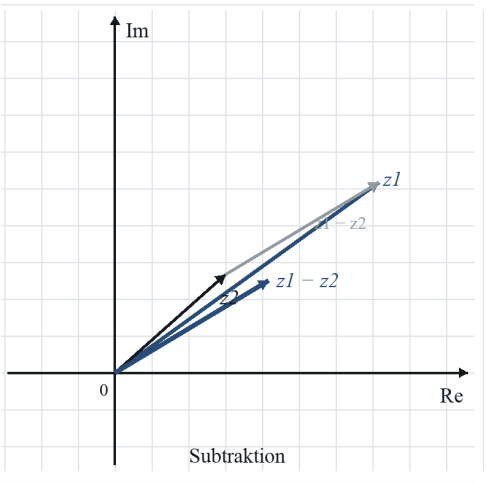


Der Punkt $z = a + bi$ wird durch seine beiden Koordinaten a und b bestimmt. Gleichwertig kann man ihn durch den Abstand $r = |z|$ vom Ursprung und den Winkel φ zur positiven reellen Achse beschreiben.

1.6.1 Addition und Subtraktion

Jede komplexe Zahl kann als Ortsvektor aufgefasst werden. Ergänzt man die Vektoren zu einem Parallelogramm, so erhält man die Summe zweier komplexer Zahlen. Die Differenz $z_1 - z_2$ entsteht durch Addition von z_1 und $-z_2$.





Bei der Addition entsteht $z_1 + z_2$ als Diagonale des Parallelogramms. Die Differenz $z_1 - z_2$ kann als Verbindungsvektor von z_2 nach z_1 gedeutet werden.

1.6.2 Multiplikation und Division

Für $z \neq 0$ führen wir Polarkoordinaten ein. Mit $r = |z| > 0$ und einem Argument φ gilt

$$z = r(\cos\varphi + i\sin\varphi).$$

Für

$$z_1 = r_1(\cos\varphi_1 + i\sin\varphi_1), \qquad z_2 = r_2(\cos\varphi_2 + i\sin\varphi_2)$$

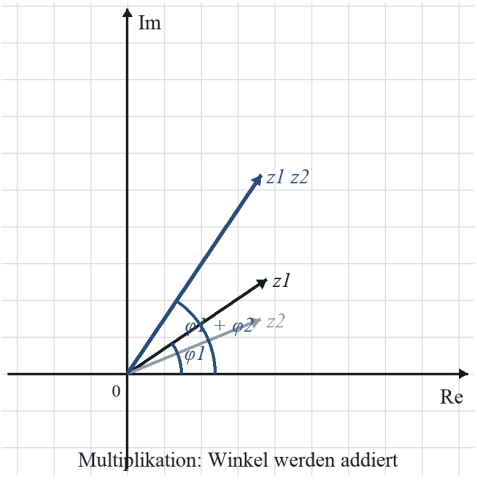
folgt aus den Additionstheoremen

$$z_1 z_2 = r_1 r_2\big(\cos(\varphi_1 + \varphi_2) + i\sin(\varphi_1 + \varphi_2)\big).$$

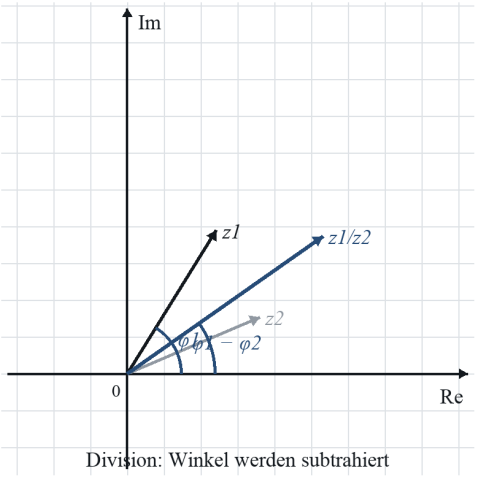
Bei der Multiplikation werden also die Beträge multipliziert und die Argumente addiert. Für $z_2 \neq 0$ gilt entsprechend

$$\frac{z_1}{z_2} = \frac{r_1}{r_2}\big(\cos(\varphi_1 - \varphi_2) + i\sin(\varphi_1 - \varphi_2)\big).$$

Bei der Division werden die Beträge dividiert und die Argumente subtrahiert.



Multiplikation: Winkel werden addiert



Division: Winkel werden subtrahiert

Die Zeichnungen veranschaulichen nur die Winkelbeziehungen. Die Länge des Produktvektors beziehungsweise des Quotientenvektors hängt zusätzlich von den Beträgen $r_1 r_2$ beziehungsweise r_1/r_2 ab.

1.7 Anordnung von komplexen Zahlen

Die komplexen Zahlen lassen sich nicht so anordnen, dass sie zusammen mit den üblichen Rechenoperationen einen geordneten Körper bilden. Wäre $i > 0$, so wäre wegen der Verträglichkeit mit der Multiplikation $i^2 = -1 > 0$. Wäre dagegen $i < 0$, so wäre $-i > 0$ und damit ebenfalls $(-i)^2 = -1 > 0$. Beides widerspricht $1 > 0$.

Als bloße Menge kann $\mathbb{C}$ selbstverständlich geordnet werden, etwa lexikographisch nach Real- und Imaginärteil. Auch eine Sortierung nach Betrag und anschließend nach einem fest gewählten

Argument ist möglich. Solche Ordnungen sind jedoch nicht mit Addition und Multiplikation in der Weise verträglich, wie es die übliche Ordnung der reellen Zahlen ist.

> **Korrekturhinweis** Der ursprüngliche Beweis betrachtete nur den Fall $i > 0$, und die anschließende Ordnung nach Betrag und Winkel wurde nicht vollständig festgelegt. Beide Lehrerhinweise sind hier berücksichtigt.

1.8 Beispiele

Beispiel 1 Die Gleichung $z^2 + 2 = 0$ besitzt die beiden Lösungen

$$z_1 = i\sqrt{2}, \qquad z_2 = -i\sqrt{2}.$$

Beispiel 2 Für $z^4 + 3 = 0$, also $z^4 = -3$, schreiben wir

$$-3 = 3\big(\cos(180^\circ) + i\sin(180^\circ)\big).$$

Mit dem Satz von de Moivre erhält man die vier Lösungen

$$z_k = \sqrt[4]{3}\left(\cos\frac{180^\circ + 360^\circ k}{4} + i\sin\frac{180^\circ + 360^\circ k}{4}\right), \qquad k = 0,1,2,3.$$

Ihre Argumente sind 45°, 135°, 225° und 315°.

Beispiel 3 Für $z^5 + 4 = 0$, also $z^5 = -4$, gilt entsprechend

$$z_k = \sqrt[5]{4}\left(\cos\frac{180^\circ + 360^\circ k}{5} + i\sin\frac{180^\circ + 360^\circ k}{5}\right), \qquad k = 0,1,2,3,4.$$

Die fünf Argumente sind 36°, 108°, 180°, 252° und 324°. Der im Original stehende Wert 342° war ein Schreibfehler.

2 Folgen und Grenzwerte in $\mathbb{C}$

2.1 Folgen und Konvergenz in $\mathbb{C}$

Eine komplexe Zahlenfolge ist eine Abbildung

$$\mathbb{N} \longrightarrow \mathbb{C}, \qquad n \longmapsto z_n.$$

Schreibt man $z_n = a_n + ib_n$, so bestehen Real- und Imaginärteile jeweils aus reellen Zahlenfolgen. Eine komplexe Folge heißt konvergent, wenn die Folge der Realteile und die Folge der Imaginärteile konvergieren.

2.1.1 Definition des Grenzwerts

Die Folge (z_n) konvergiert gegen $z \in \mathbb{C}$, wenn zu jedem $\varepsilon > 0$ ein $N \in \mathbb{N}$ existiert, sodass

$$|z_n - z| < \varepsilon \qquad \text{für alle } n \geq N$$

gilt. Man schreibt

$$\lim_{n \to \infty} z_n = z.$$

Für $z_n = a_n + ib_n$ und $z = a + ib$ gilt

$$z_n \longrightarrow z \quad \Longleftrightarrow \quad a_n \longrightarrow a \ \text{ und } \ b_n \longrightarrow b.$$

Aus $|z_n - z| < \varepsilon$ folgen unmittelbar $|a_n - a| < \varepsilon$ und $|b_n - b| < \varepsilon$. Umgekehrt wählt man für beide reellen Folgen die Schranke $\varepsilon/2$; dann gilt

$$|z_n - z| \leq |a_n - a| + |b_n - b| < \varepsilon.$$

> **Korrekturhinweis** Im Original wurde nur eine Richtung dieser Äquivalenz bewiesen. Die von der Lehrkraft verlangte Rückrichtung ist ergänzt. Die Grenzwertdefinition ist außerdem vom eigentlichen Cauchy-Kriterium zu unterscheiden.

2.2 Grenzwertsätze

Seien $z_n \to z$ und $w_n \to w$. Dann gelten

$$z_n + w_n \longrightarrow z + w,$$
$$z_n - w_n \longrightarrow z - w,$$
$$z_n w_n \longrightarrow zw.$$

Falls $w \neq 0$, ist w_n ab einem hinreichend großen Index ebenfalls ungleich null, und es gilt

$$\frac{z_n}{w_n} \longrightarrow \frac{z}{w}.$$

Wegen der Stetigkeit des Betrags folgt außerdem

$$|z_n| \longrightarrow |z|.$$

Die im Original angeschlossene Aussage über Polarkoordinaten benötigt eine Einschränkung: Ist $z \neq 0$ und wird in einer Umgebung von z ein stetiger Zweig des Arguments gewählt, dann konvergieren auch die entsprechend gewählten Argumente. Ohne eine solche Festlegung sind Winkel nur bis auf Vielfache von 2π bestimmt.

2.3 Beispiel

Gegeben sei

$$z_n = \frac{n^2 i^n}{n^3 + 1}.$$

Da $|i^n| = 1$, gilt

$$|z_n| = \frac{n^2}{n^3 + 1} \leq \frac{1}{n} \longrightarrow 0.$$

Somit konvergiert z_n gegen 0. Für $\varepsilon > 0$ genügt die Wahl eines $N > 1/\varepsilon$, denn dann ist für alle $n \geq N$

$$|z_n| < \varepsilon.$$

Geometrisch bedeutet dies, dass ab diesem Index alle Folgenglieder innerhalb des Kreises mit Mittelpunkt 0 und Radius ε liegen.

> **Korrekturhinweis** Der ursprüngliche Lösungsweg behandelte Zähler und Nenner getrennt, obwohl deren einzelne Grenzwerte nicht als endliche Zahlen existieren, und kehrte eine ε-Ungleichung falsch um. Die direkte Betragsabschätzung schließt diese Lücke, ohne das Beispiel auszutauschen.

3 Die Mandelbrot-Menge

3.1 Allgemeines

Die Mandelbrot-Menge, vereinfacht auch „Apfelmännchen“ genannt, wurde um 1980 durch Benoît B. Mandelbrot weithin bekannt und nach ihm benannt. Mandelbrot arbeitete am Thomas-J.-Watson-Forschungszentrum der IBM in Yorktown Heights bei New York. Sein Buch über die fraktale Geometrie der Natur machte die neuen Formen auch außerhalb eines kleinen Fachkreises bekannt.

Die Mathematik befasste sich zunehmend mit komplexen Rückkopplungsprozessen. Solche Prozesse sind im Prinzip schon lange bekannt und lassen sich in einfachen Fällen durch Differentialgleichungen beschreiben. Durch Mandelbrots Arbeiten wurde sichtbar, dass sich mit iterierten Gleichungen auch gebrochene, unregelmäßige Strukturen untersuchen lassen, die an Formen in der Natur erinnern, etwa Wolken oder Farnblätter. Die Berechnung der zugehörigen Bilder wurde durch schnelle Rechenanlagen möglich.

Eine Iteration verwendet das Ergebnis eines Rechenschritts als Eingabe des nächsten Schritts. Allgemein schreibt man

$$x_0 = k, \qquad x_{n+1} = f(x_n).$$

Damit erhält man

$$x_1 = f(k), \qquad x_2 = f(f(k)), \qquad x_3 = f(f(f(k))), \quad \ldots$$

Die Schreibweise $f^2(x)$ bezeichnet hier die zweimalige Anwendung $f(f(x))$ und darf nicht mit $(f(x))^2$ verwechselt werden. Dieses Verfahren wird als Iteration oder Rückkopplung bezeichnet.

3.2 Mathematische Erfassung der Mandelbrot-Menge

Wir versuchen nun in diesem Kapitel, eine komplexe Funktion vorzustellen, die uns den Weg in die Welt der ästhetischen Computergrafiken eröffnet. Im Prinzip lassen sich viele komplexe Funktionen iterieren. Für unser Beispiel, die Apfelmännchen-Grafik, wählen wir die quadratische Funktion

$$f_k(z) = z^2 + k, \qquad z = a + bi, \quad k = c + di, \quad a,b,c,d \in \mathbb{R}.$$

Für Real- und Imaginärteil folgt

$$f_k(a + bi) = (a^2 - b^2 + c) + i(2ab + d).$$

Jeder Punkt der komplexen Ebene wird durch eine komplexe Zahl beschrieben; sein Abstand vom Ursprung ist ihr Betrag. Für eine feste Konstante k wird die Funktion, ausgehend vom Startwert null, wiederholt angewendet:

$$z_0 = 0, \qquad z_{n+1} = z_n^2 + k.$$

Die ersten Iterationen lauten damit

$$z_1 = k, \qquad z_2 = k^2 + k, \qquad z_3 = (k^2 + k)^2 + k, \quad \ldots$$

Für die spätere Bildkonstruktion ist entscheidend, ob diese Folge beschränkt bleibt oder schließlich ins Unendliche entweicht.

Korrekturhinweis Im Original wird trotz des Startwertes $z_0 = 0$ der Ausdruck $k^2 + k$ als erster Schritt notiert. Er ist tatsächlich das zweite Folgenglied z_2. Die Rechnung wurde deshalb nur neu indiziert; der ursprüngliche Ansatz bleibt erhalten.

Drei Fälle für den Sonderfall $k = 0$

Die folgende Fallunterscheidung des Originals ist für die besondere Abbildung $f_0(z) = z^2$, also für $k = 0$, richtig. Gemeint sind dabei stets die Beträge der komplexen Zahlen.

a) Ist $|z| < 1$, so gilt $|z^{2^n}| = |z|^{2^n} \to 0$. Die Folge nähert sich dem Nullpunkt.

b) Ist $|z| = 1$, so haben alle Folgenglieder den Betrag 1. Die Folgenglieder selbst müssen jedoch nicht gleich sein.

c) Ist $|z| > 1$, so wächst $|z|^{2^n}$ unbegrenzt; die Folge entweicht gegen unendlich.

Für die Mandelbrot-Menge wird aber $f_k(z) = z^2 + k$ untersucht. Hier darf der Summand k nicht fortgelassen werden. Deshalb lässt sich die Zugehörigkeit nicht allein durch die drei Fälle am Einheitskreis entscheiden. Ein einfaches Gegenbeispiel ist $k = -1$:

$$0 \longrightarrow -1 \longrightarrow 0 \longrightarrow -1 \longrightarrow \cdots$$

Die Bahn bleibt beschränkt, obwohl sie nicht gegen null konvergiert.

Lehrerhinweis Die rote Frage „Wieso entfällt das k?“ bezeichnet genau die Lücke im ursprünglichen Übergang. Die Fallunterscheidung wird deshalb nicht gestrichen, sondern ausdrücklich als Sonderfall $k = 0$ erhalten.

3.3 Der „Apfelmännchen“-Algorithmus

Folgen, die gegen einen festen Wert oder gegen eine periodische Folge streben, werden von einem Attraktor angezogen. Entweichen die Beträge dagegen ohne Grenze, kann man anschaulich vom Attraktor unendlich sprechen. Für die Mandelbrot-Menge ist jedoch nicht vorgeschrieben, dass die Folge gegen null konvergiert. Entscheidend ist, ob die bei null beginnende Bahn beschränkt bleibt:

$$\mathcal{M} = \left\{ k \in \mathbb{C} \;\middle|\; z_0 = 0, \quad z_{n+1} = z_n^2 + k, \quad (z_n) \text{ ist beschränkt} \right\}.$$

Die Berechnung einzelner Folgen von Hand kann das Verhalten nur an wenigen Beispielen zeigen. Um einen größeren Ausschnitt der komplexen Ebene zu untersuchen, überträgt man die wiederholte Rechnung auf den Computer. Dazu sind im Wesentlichen drei Schleifen erforderlich: zwei Schleifen durchlaufen die Bildpunkte, die dritte führt für jeden Bildpunkt die Iteration aus.

Korrekturhinweis Ersetzt wurde nur das falsche Zugehörigkeitskriterium. Im Original wurde die Mandelbrot-Menge über die Konvergenz $f^n(z) \to 0$ definiert. Richtig ist die Beschränktheit der kritischen Bahn; der untersuchte Punkt ist der Parameter k, nicht ein beliebiger Startwert.

Ausführung des Algorithmus

Wir wählen beispielsweise den Bereich $-2 \le c \le 1$ und $-1{,}5 \le d \le 1{,}5$ der komplexen Ebene. Jedem Bildpunkt wird eine Konstante $k = c + di$ zugeordnet. Die Variablen a und b enthalten während der Rechnung Real- und Imaginärteil des jeweils aktuellen Folgengliedes.

1. Bestimme aus der Bildgröße und dem gewählten Ausschnitt die Schrittweiten für c und d.
2. Ordne dem aktuellen Bildpunkt den Parameter $k = c + di$ zu.
3. Setze $a := 0$, $b := 0$ und den Iterationszähler $h := N$.
4. Berechne
$$\mathrm{Re} := a^2 - b^2 + c, \qquad \mathrm{Im} := 2ab + d.$$
5. Setze $a := \mathrm{Re}$, $b := \mathrm{Im}$ und $h := h - 1$.
6. Wiederhole die Rechnung, bis $a^2 + b^2 > 4$ oder $h = 0$ gilt.
7. Ist $h = 0$, so wird der Bildpunkt als innerhalb des Iterationslimits nicht entflohen gekennzeichnet. Andernfalls ist er sicher kein Punkt der Mandelbrot-Menge.

Der so erzeugte Bildausschnitt ist eine Näherung. Bei einer endlichen Zahl von Iterationen bedeutet ein eingefärbter Punkt nicht, dass die Zugehörigkeit bewiesen ist, sondern nur, dass die Folge bis dahin nicht entwichen ist.

> **Korrekturhinweis** Im ursprünglichen Schema werden Real- und Imaginärteil berechnet, aber nicht wieder den Variablen a und b zugewiesen. Ohne diese Aktualisierung findet keine echte Iteration statt. Die beiden Zuweisungen wurden ergänzt; das erhaltene Pascal-Listing führt sie ebenfalls aus.

Fluchtradius und Julia-Mengen

Je nach Rechengeschwindigkeit kann die maximale Zahl der Iterationen beispielsweise zwischen 25 und 150 gewählt werden. Eine größere Zahl verbessert die Näherung besonders am Rand, erhöht aber die Rechenzeit.

Für die kritische Bahn genügt der Radius 2 als Fluchtgrenze. Ist $|k| > 2$, so liegt bereits $z_1 = k$ außerhalb dieses Kreises. Für $|k| \le 2$ und einen späteren Bahnpunkt mit $|z_n| > 2$ folgt

$$|z_{n+1}| = |z_n^2 + k| \ge |z_n|^2 - |k| \ge |z_n|^2 - 2 > |z_n|.$$

Der Betrag wächst danach weiter und die Bahn entweicht gegen unendlich. Die Gleichheit $|z_n| = 2$ ist für sich allein jedoch kein einheitlicher Sonderfall. Bei $k = -2$ erreicht die kritische Bahn den Wert 2 und bleibt dort; daraus folgt keine allgemeine Aussage für alle Parameter oder Bahnpunkte mit Betrag 2.

Zum Vergleich betrachten wir nun die Julia-Mengen. Mandelbrot- und Julia-Mengen entstehen aus derselben Funktionsschar, aber durch zwei verschiedene Fragestellungen:

- Bei der Mandelbrot-Menge wird der Parameter k verändert, während der Startwert $z_0 = 0$ fest bleibt.
- Für eine gefüllte Julia-Menge K_k wird k festgehalten und der Startpunkt z_0 variiert.

Zwischen beiden besteht der wichtige Zusammenhang

$$k \in \mathcal{M} \quad \Longleftrightarrow \quad K_k \text{ ist zusammenhängend.}$$

Dabei bedeutet „zusammenhängend“ nicht, dass sich je zwei Punkte zwingend durch einen Polygonzug innerhalb der Menge verbinden lassen. Diese Eigenschaft wäre stärker als die hier benötigte Zusammenhängendheit.

> **Historische Einordnung** Die Trennung von Parameter- und Dynamikebene sowie der Zusammenhang zwischen $k \in \mathcal{M}$ und der Zusammenhängendheit von K_k waren 1990 Bestandteil der Theorie. Sie sind historische Korrekturen und keine nachträglich angelegten modernen Maßstäbe.

3.4 Verschiedene Darstellungsarten von Mandelbrot-Mengen

Mandelbrot-Mengen lassen sich auf viele Arten darstellen. Je nach Wahl der Konfiguration, etwa als Farb- oder Schwarz-Weiß-Darstellung, können unterschiedliche Effekte erzielt werden. Wird der Iterationsprozess abgebrochen, sobald der Betrag die vorgegebene Fluchtgrenze überschreitet, kann man der Anzahl der bis dahin benötigten Iterationen eine Farbe oder einen Grauwert zuordnen. Dadurch erhält man unter anderem folgende Möglichkeiten:

a) Es werden nur die Punkte markiert, deren Bahn innerhalb des Iterationslimits nicht entweicht.

b) Punkte mit gleicher Fluchtzeit erhalten dieselbe Farbe. So entstehen farbige Bänder, die an Höhenlinien erinnern.

c) Kombiniert man die beiden Verfahren, erhält man eine differenzierte Darstellung mit vielen Abstufungen.

d) Die Fluchtgeschwindigkeit kann als Höhe interpretiert werden. Dadurch entsteht eine dreidimensional wirkende Darstellung, etwa ein „Mandelbrot-See“ mit umliegenden Hochflächen.

e) Ordnet man jedem Iterationsschritt eine andere Farbe zu, entstehen besonders farbige Bilder.

f) Ersetzt man Farben durch Töne, kann aus den Iterationsdaten eine musikalische Komposition entstehen.

Darüber hinaus sind viele weitere Darstellungsarten und Variationen möglich.

3.5 Ähnliche Strukturen innerhalb der Mandelbrot-Menge

Ein weiterer Aspekt, der unmittelbar mit der Mandelbrot-Menge zusammenhängt, ist ihre annähernde Selbstähnlichkeit beziehungsweise Skaleninvarianz. Selbstähnlich nennt man Gebilde, die bei Vergrößerung in verschiedenen Größenordnungen ähnliche Strukturen zeigen.

Untersucht man kleine Ausschnitte am Rand der Mandelbrot-Menge und vergrößert sie, erscheinen immer neue Einzelheiten. Dabei treten Miniaturen der Apfelmännchenform auf, die mit der Hauptfigur verwandt sind. Die Mandelbrot-Menge ist jedoch nicht überall exakt selbstähnlich; ihre Wiederholungen sind verzerrt und von neuen Strukturen umgeben.

Solche selbstähnlichen oder annähernd selbstähnlichen Muster findet man auch in der Natur, etwa in Verzweigungen von Bäumen, Farnen oder Flusssystemen.

3.6 Weitere Erkenntnisse über Mandelbrot-Mengen

Die in dieser Arbeit untersuchte quadratische Funktion ist der einfachste Fall. Auch Polynome höheren Grades können iteriert werden und erzeugen eigene Parameter- und Dynamikmengen. Bei Polynomen dritten Grades müssen im Allgemeinen mehrere kritische Punkte berücksichtigt werden. Zusätzliche komplexe Parameter können zu höherdimensionalen Parameterräumen führen; eine einzelne komplexe Variable bleibt dagegen in der zweidimensionalen komplexen Ebene darstellbar.

Viele Darstellungen besitzen Symmetrien. Andere Strukturen zeigen einen empfindlichen Übergang zwischen Ordnung und chaotischem Verhalten. Die Chaostheorie untersucht, wie aus einfachen deterministischen Vorschriften sehr komplizierte Verläufe entstehen können.

Fraktale Geometrie ist nicht nur Gegenstand theoretischer Forschung. Ihre Grundideen können auch in Unterricht und Ausbildung vermittelt werden, sofern zwischen anschaulichen Bildern, numerischen Näherungen und mathematisch bewiesenen Aussagen unterschieden wird.

4 Fraktale Dimension

4.1 Die Kochsche Schneeflocke

Die klassische euklidische Geometrie beschreibt im Wesentlichen glatte Formen wie Linien, Kreise, Rechtecke, Kugeln und Pyramiden. Viele Formen in der Natur sind jedoch komplizierter. Ein bekanntes Beispiel für eine fraktale Kurve ist die Kochsche Schneeflocke.

Man beginnt mit einer Strecke als Initiator. Diese wird in drei gleich lange Teile zerlegt; das mittlere Drittel wird durch zwei Seiten eines gleichseitigen Dreiecks ersetzt. Der so entstandene Streckenzug heißt Generator. Wird jeder seiner vier Abschnitte wieder nach derselben Vorschrift ersetzt und dieser Vorgang unbegrenzt fortgesetzt, entsteht die Koch-Kurve.

Der Rand wird dabei immer länger, während die entstehende Figur in einem begrenzten Gebiet bleibt. Die Kurve besitzt keine gewöhnliche ganzzahlige Liniendimension. Sie ist stärker als eine Linie verzweigt, füllt aber keine Fläche aus. Für solche gebrochenen Formen führte Benoît Mandelbrot den Begriff „Fraktal“ ein.

4.2 Fraktale Dimension

Bei einer exakt selbstähnlichen Figur bestehe jede Stufe aus N verkleinerten Kopien des Ganzen, wobei jede Kopie um den Faktor $s > 1$ verkleinert ist. Dann gilt

$$N = s^D \qquad \Longrightarrow \qquad D = \frac{\log N}{\log s}.$$

Diese Zahl D heißt Ähnlichkeitsdimension. Für eine Strecke, ein Quadrat und einen Würfel mit dem Skalierungsfaktor $s = 3$ ergeben sich $N = 3$, $N = 9$ und $N = 27$, also die Dimensionen 1, 2 und 3.

Bei der Koch-Kurve entstehen aus jedem Abschnitt vier neue Abschnitte, die jeweils um den Faktor 3 verkleinert sind. Daher gilt

$$D = \frac{\log 4}{\log 3} \approx 1{,}26186.$$

Die Dimension ist somit nicht ganzzahlig, sondern liegt zwischen der Dimension einer Linie und der Dimension einer Fläche.

Für nicht exakt selbstähnliche Mengen lässt sich die obige Formel nicht unmittelbar anwenden. Eine gebräuchliche numerische Näherung ist das Box-Counting-Verfahren: Man überdeckt die Menge mit Quadraten der Seitenlänge ε, zählt die benötigten Quadrate $N(\varepsilon)$ und untersucht das Verhältnis

$$\frac{\log N(\varepsilon)}{\log(1/\varepsilon)} \quad \text{für} \quad \varepsilon \to 0.$$

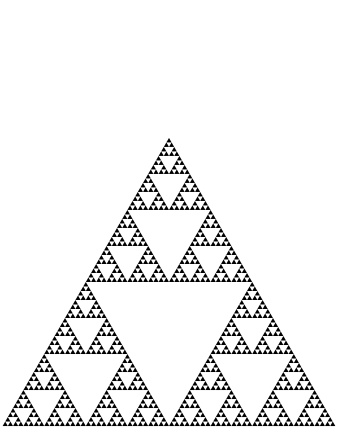
> **Historische Einordnung** Im Original blieb unklar, ob mit der Dimension die ganze Mandelbrot-Menge oder ihr Rand gemeint war. Die Mandelbrot-Menge selbst besitzt inneres Gebiet und hat daher Hausdorff-Dimension 2. Dass auch ihr Rand Hausdorff-Dimension 2 besitzt, wurde erst nach 1990 bewiesen und darf nicht rückwirkend als damals bekannte Aussage bewertet werden.

5 Anhang

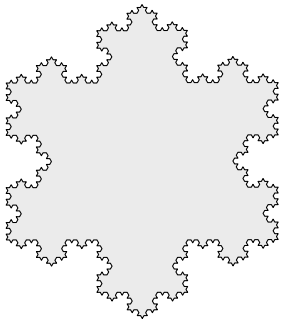
5.1 Verschiedene Bilder

Die folgenden Tafeln bewahren die ursprüngliche Bildauswahl sowie, soweit das Original sie angibt, Koordinaten und Iterationstiefen. Die mit Koordinaten versehenen Mandelbrot-Abbildungen wurden mit einem Python-Programm in Schwarz-Weiß neu berechnet. Auch die dreidimensionale Darstellung wurde aus den Fluchtzeiten neu erzeugt. Die Farbtafel wurde durch ähnliche Ausschnitte ersetzt, Koch-Kurve und Skalierungstabelle wurden in LaTeX neu gezeichnet. Die Tafel mit den IFS-Fraktalen zeigt drei neu erzeugte Beispiele. Die darunterstehenden Erläuterungen präzisieren die mathematische Bedeutung.

Tafel 1: Beispiele iterierter Funktionssysteme (Originalseite 31)



Sierpiński-Dreieck



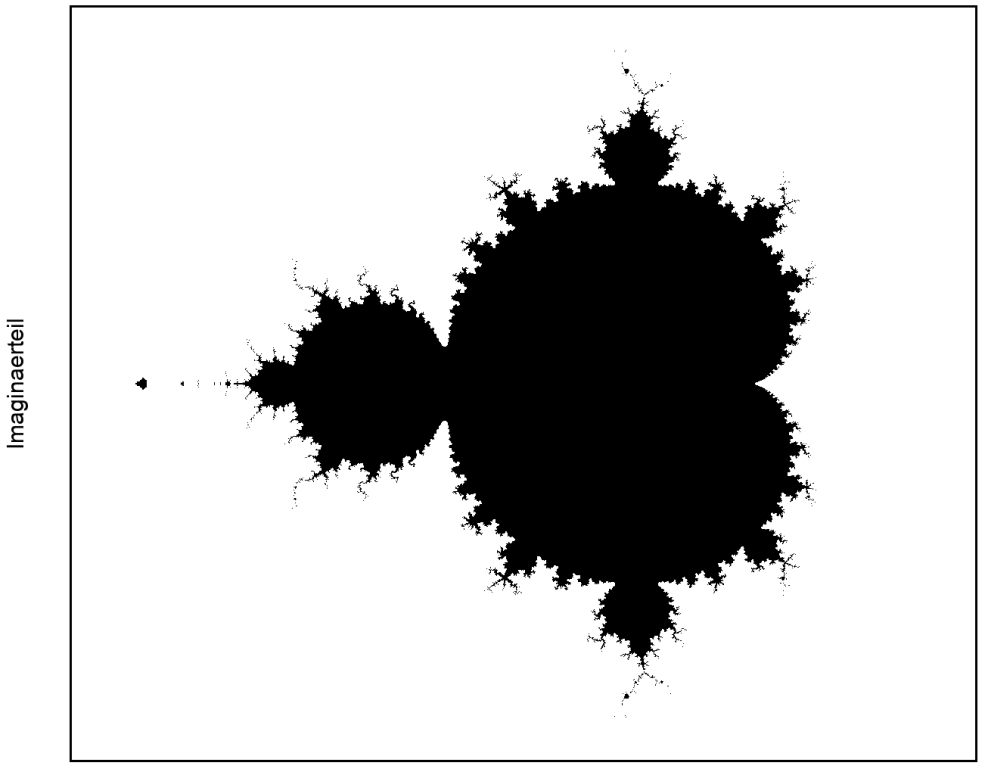
Kochsche Schneeflocke



Barnsley-Farn

Das Original zeigt verschiedene IFS-Fraktale, darunter die Kochsche Schneeflocke und das Sierpiński-Dreieck. Bei den übrigen Motiven fehlen Bezeichnungen beziehungsweise Quellenangaben. Die Abbildungen wurden 2026 neu erzeugt und beschränkt sich auf Beispiele mit bekannter Vorschrift: Sierpiński-Dreieck und Schneeflocke sind exakt konstruiert, der Farn entsteht aus den vier affinen Abbildungen nach Michael Barnsley (*Fractals Everywhere*, 1988). Diese Figuren entstehen durch iterierte Abbildungssysteme. Sie gehören nicht zur Mandelbrot-Menge, zeigen aber ebenfalls, wie aus einer einfachen, wiederholten Vorschrift komplizierte und selbstähnliche Strukturen hervorgehen können.

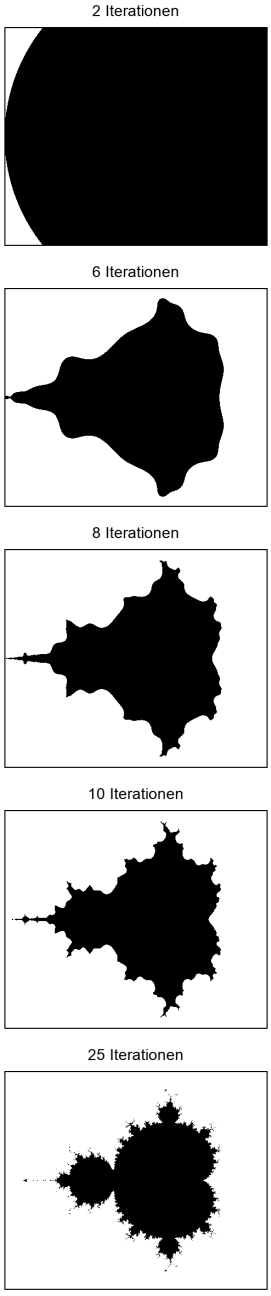
Tafel 2: Mandelbrot-Menge in der komplexen Ebene (Originalseite 32)




```
   Realteil:  Minimum -2.00000000   Maximum 1.00000000
Imaginaerteil:  Minimum -1.25000000   Maximum 1.25000000
                     Rechentiefe: 25
```

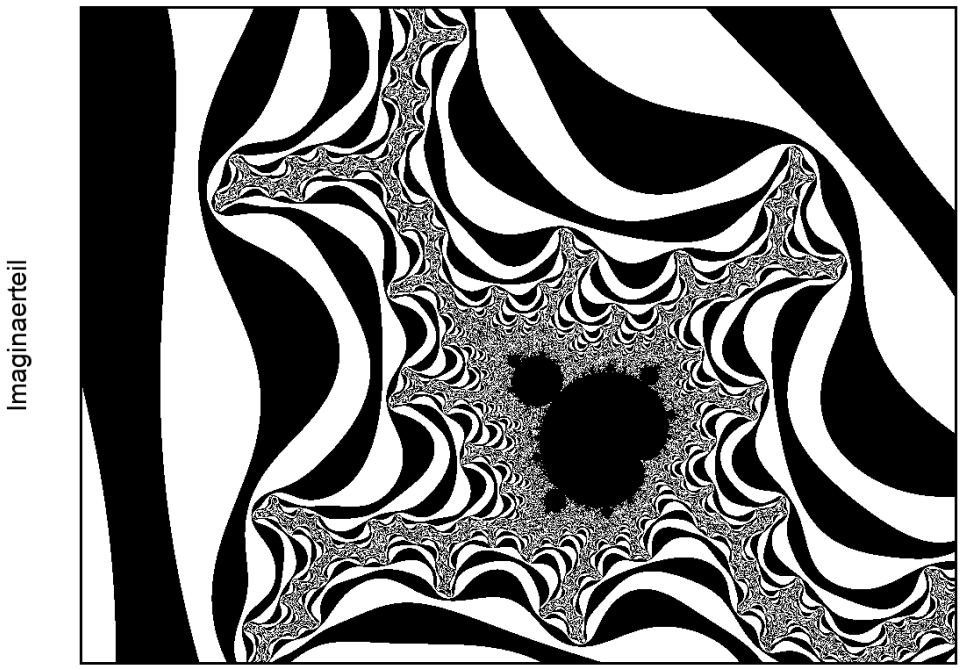
Die schwarzen Punkte sind nach 25 Iterationen nicht entflohen. Damit handelt es sich um eine numerische Näherung der Mandelbrot-Menge. Ein schwarzer Punkt muss deshalb nicht endgültig zur Mandelbrot-Menge gehören; seine Folge kann auch erst nach mehr als 25 Schritten die Fluchtgrenze überschreiten. Konvergenz gegen null wird nicht vorausgesetzt.

Tafel 3: Wirkung des Iterationslimits (Originalseite 33)



Die Bildfolge zeigt die Näherung nach 2, 6, 8, 10 und 25 Iterationen. Mit steigendem Iterationslimit werden immer mehr außerhalb liegende Punkte als entweichend erkannt. Die Mandelbrot-Menge selbst verändert sich dabei nicht. Es wird lediglich die äußere Näherung genauer. Besonders am Rand sind deshalb bei größerer Rechentiefe immer feinere Einzelheiten zu erkennen.

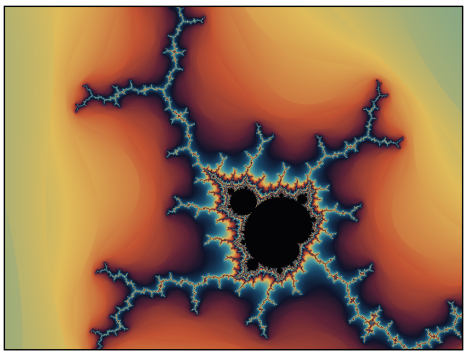
Tafel 4: Vergrößerung eines Randgebiets (Originalseite 34)



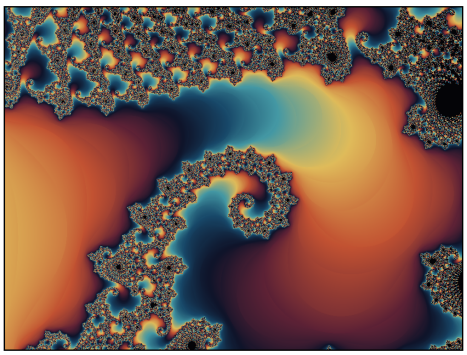

```
  Realteil:  Minimum -0.19920000   Maximum -0.12954000
Imaginaerteil:  Minimum 1.01480000   Maximum 1.06707000
                    Rechentiefe: 200
```

Die Abbildung zeigt eine Vergrößerung am Rand der Mandelbrot-Menge. Durch weitere Vergrößerung treten immer neue Strukturen und Miniaturen hervor. Der dargestellte Bereich hat im Realteil nur noch eine Breite von 0,06966. Die schwarzen und weißen Bänder bezeichnen unterschiedliche Fluchtzeiten; sie sind keine zusätzlichen Bestandteile der Mandelbrot-Menge.

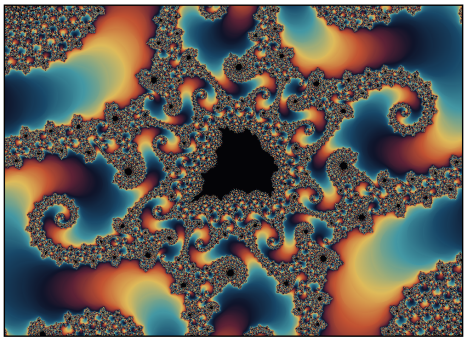
Tafel 5: Farbige Fraktalabbildungen (Originalseite 35)



Realteil: Minimum −0.19920000 Maximum −0.12954000
Imaginaerteil: Minimum 1.01480000 Maximum 1.06707000
Rechentiefe: 260



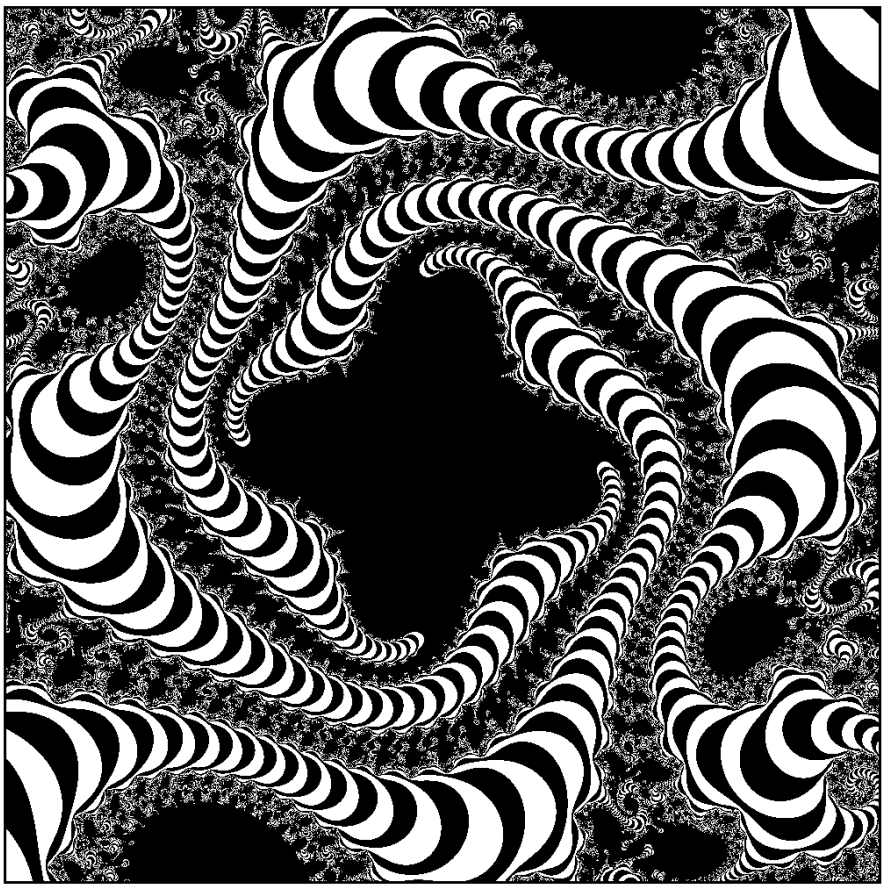
Realteil: Minimum −0.74700000 Maximum −0.74000000
Imaginaerteil: Minimum 0.12900000 Maximum 0.13425000
Rechentiefe: 320



Realteil: Minimum −0.74543560 Maximum −0.74542150
Imaginaerteil: Minimum 0.11300370 Maximum 0.11301390
Rechentiefe: 450

Das Originalfarbblatt ist im Scan `IMG_1960` erhalten, nennt jedoch keine Koordinaten. Für diese Lesefassung wurden deshalb drei ähnliche Ausschnitte mit einem Python-Programm neu berechnet; eine genaue Rekonstruktion der ursprünglichen Bilder wird damit nicht beansprucht. Unter jedem Bild sind der verwendete Koordinatenbereich und die Rechentiefe angegeben. Die Farben stellen Gruppen ähnlicher Fluchtzeiten dar. Sie gehören nicht zur Mandelbrot-Menge als mathematischer Punktmenge. Eine andere Farbzuordnung würde daher das Aussehen verändern, nicht aber die berechneten Folgen.

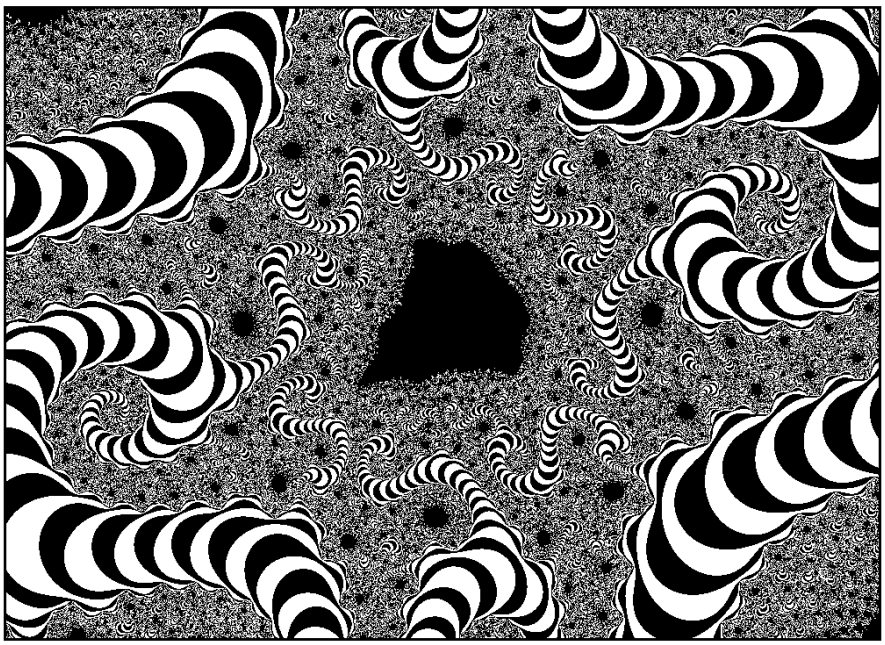
Tafel 6: Schwarz-Weiß-Neuberechnung (Originalseite 36)




```
   Realteil:  Minimum -0.74638270   Maximum -0.74638210
Imaginaerteil:  Minimum -0.09850600   Maximum -0.09850540
                     Rechentiefe: 350
```

Eine starke Vergrößerung mit 350 Iterationen. Die schmalen schwarzen Verbindungen und wiederkehrenden Miniaturen veranschaulichen die feine Randstruktur. Die Breite des Ausschnitts beträgt im Realteil nur 0,00000060. Die Wiederholungen sind der Hauptfigur ähnlich, aber nicht mit ihr identisch, da in jeder Vergrößerungsstufe neue Strukturen hinzukommen.

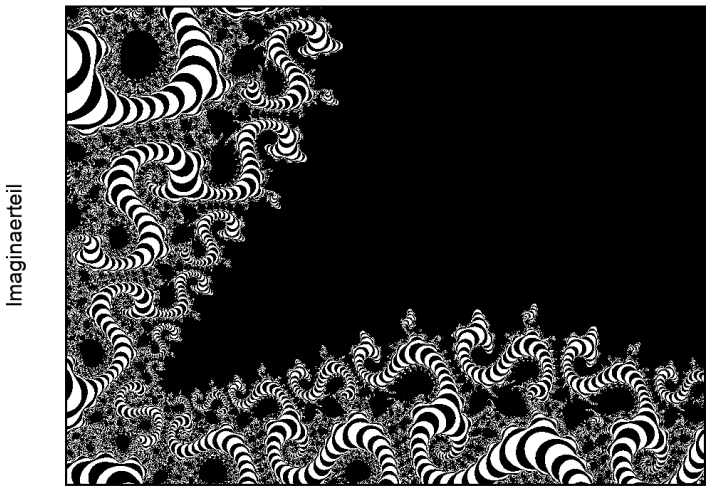
Tafel 7: Weitere Miniatur (Originalseite 37)




```
  Realteil:  Minimum -0.74543560   Maximum -0.74542150
Imaginaerteil:  Minimum 0.11300370   Maximum 0.11301390
                    Rechentiefe: 350
```

Auch dieser Ausschnitt zeigt eine angenäherte Symmetrie und eine punktähnliche Miniatur der Hauptfigur. Seine Breite im Realteil beträgt 0,00001410. Die gesamte Mandelbrot-Menge ist genau symmetrisch zur reellen Achse. Die hier sichtbare örtliche Symmetrie ist dagegen nur näherungsweise vorhanden.

Tafel 8: Zwei Ausschnittsvergrößerungen (Originalseite 38)



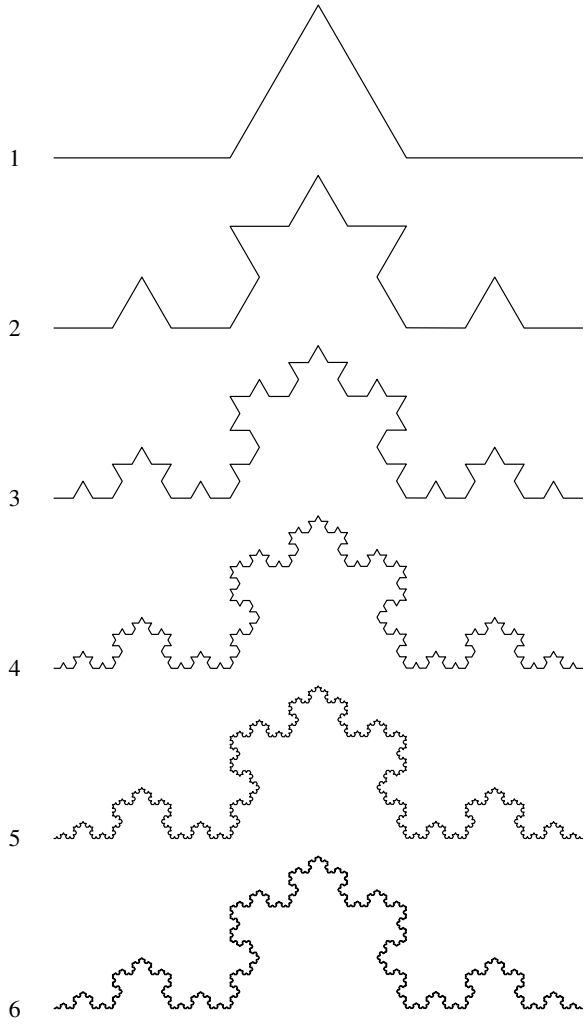
Realteil: Minimum −0.74543010 Maximum −0.74542890
Imaginaerteil: Minimum 0.11300760 Maximum 0.11300850
Rechentiefe: 350

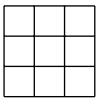
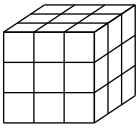


Realteil: Minimum −0.75104000 Maximum −0.74080000
Imaginaerteil: Minimum 0.10835000 Maximum 0.11536000
Rechentiefe: 300

Die Abbildungen dokumentieren unterschiedliche Ausschnitte und Rechentiefen der damaligen Computerdarstellung. Der obere Ausschnitt umfasst im Realteil nur 0,0000012, der untere 0,01024. Die Bänder entstehen durch die abwechselnd schwarze und weiße Darstellung der Fluchtzeiten. Sie machen die Geschwindigkeit des Entweichens sichtbar, beweisen aber nicht die Zugehörigkeit eines einzelnen Punktes zur Mandelbrot-Menge.

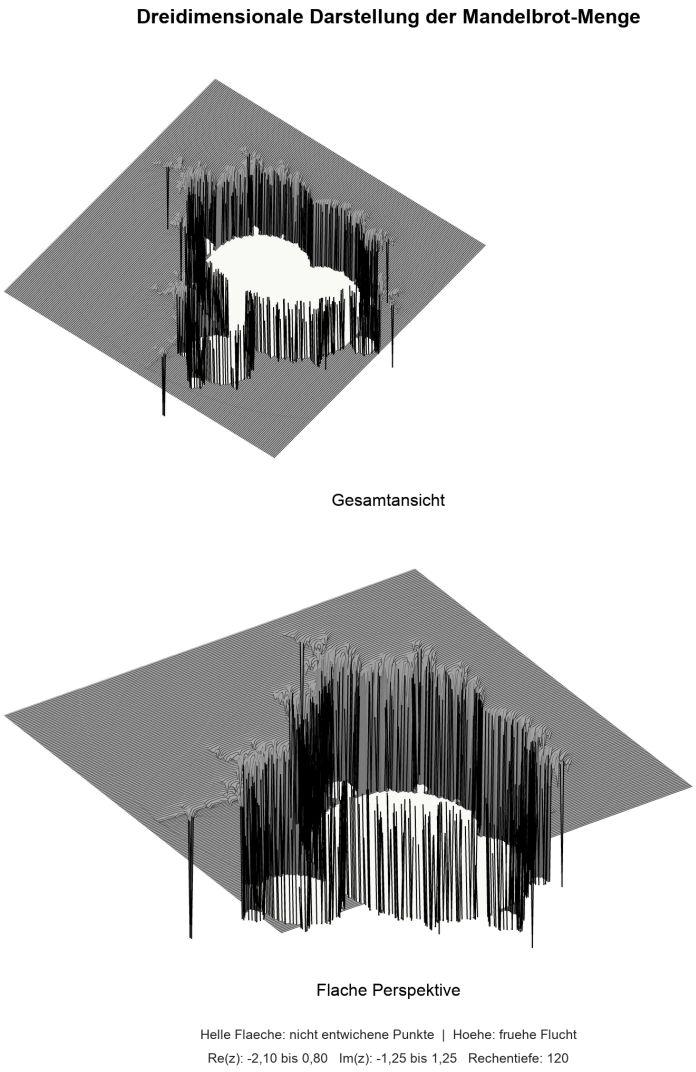
Tafel 9: Koch-Kurve und Skalierung (Originalseite 39)



	Form	Anzahl N	Skalierung s	Gesetz
	Strecke	3	3	$3^1 = 3$
	Quadrat	9	3	$3^2 = 9$
	Würfel	27	3	$3^3 = 27$

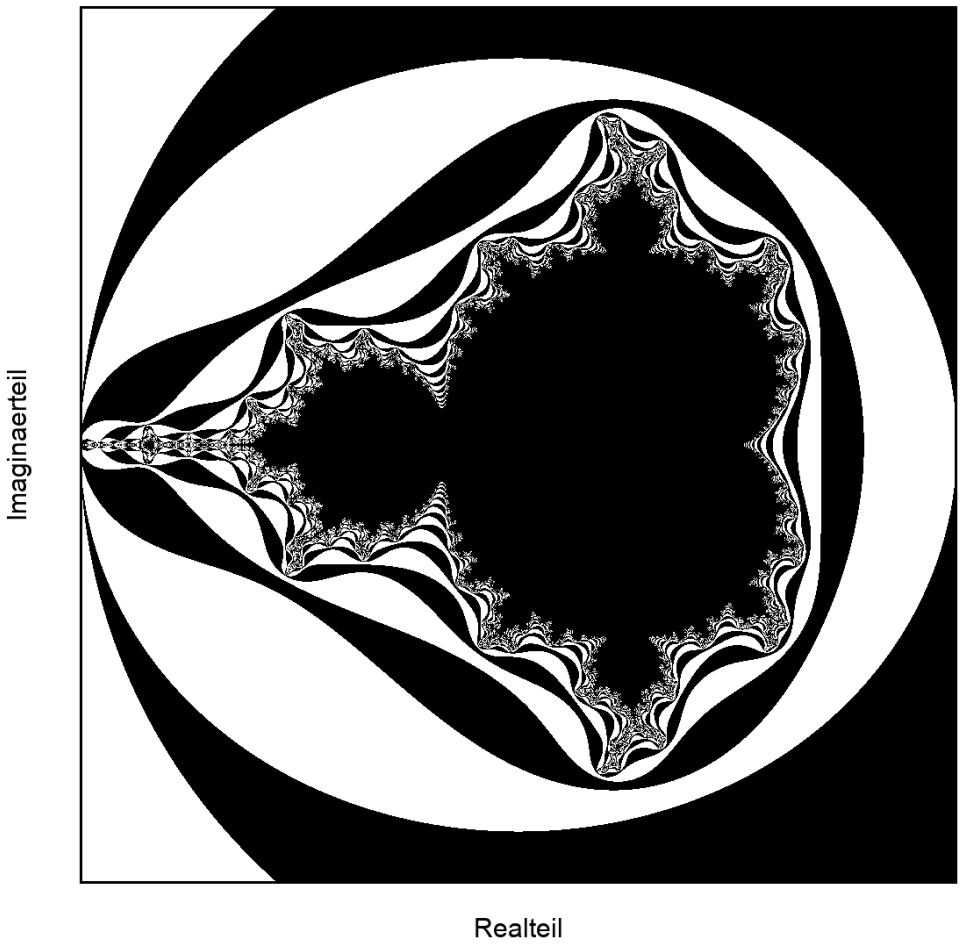
Die Iterationsstufen der Koch-Kurve und die Tabelle zu Strecke, Quadrat und Würfel verdeutlichen das Skalierungsgesetz $N = s^D$. Kurve und Tabelle wurden 2026 nach dem Original neu gezeichnet; die Bezeichnungen a und d des Originals sind dabei an die Schreibweise $N = s^D$ aus Abschnitt 4.2 angeglichen. Die Koch-Kurve wird nach einer anderen Vorschrift als die Mandelbrot-Menge erzeugt. Sie dient hier als übersichtliches Beispiel, an dem sich Selbstähnlichkeit und eine gebrochene Dimension unmittelbar berechnen lassen.

Tafel 10: Dreidimensionale Darstellung (Originalseite 40)



Die beiden Perspektiven wurden für $\mathrm{Re}(c) \in [-2{,}10; 0{,}80]$, $\mathrm{Im}(c) \in [-1{,}25; 1{,}25]$ und eine Rechentiefe von 120 Iterationen neu berechnet; das Original nennt für diese Tafel keine Koordinaten, die Werte wurden 2026 gewählt. Punkte, deren Bahn innerhalb dieses Limits nicht entweicht, bilden die helle Fläche des „Mandelbrot-Sees“. Bei den übrigen Punkten wird die Höhe umso größer gewählt, je früher die Bahn den Fluchtradius überschreitet. Die Höhe ist daher keine dritte räumliche Koordinate der Mandelbrot-Menge, sondern eine Visualisierung numerischer Daten. Je nach Höhenfunktion kann dieselbe Berechnung verschieden aussehende Landschaften ergeben; die ebene Mandelbrot-Menge bleibt unverändert.

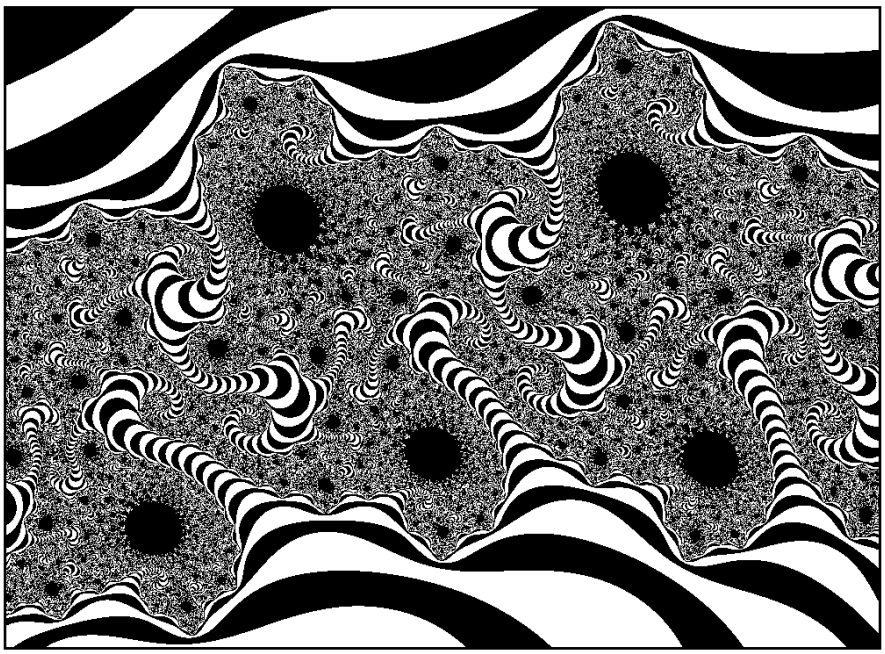
Tafel 11: Differenzierte grafische Darstellung (Originalseite 41)




```
   Realteil:  Minimum -2.00000000   Maximum 1.00000000
Imaginaerteil:  Minimum -1.50000000   Maximum 1.50000000
                    Rechentiefe: 25
```

Die Darstellung verwendet die in Abschnitt 3.4 erläuterte differenzierte Färbung beziehungsweise Abstufung. Punkte mit ähnlicher Fluchtzeit werden zu gemeinsamen Bändern zusammengefasst. Die äußeren Ringe und Bögen zeigen somit nicht den Rand der Mandelbrot-Menge allein, sondern auch das Verhalten der entweichenden Folgen in ihrer Umgebung.

Tafel 12: Ausschnitt mit hoher Rechentiefe (Originalseite 42)




```
  Realteil:  Minimum -0.74553800   Maximum -0.74505400
Imaginaerteil:  Minimum 0.11288100   Maximum 0.11323600
                    Rechentiefe: 250
```

Der Ausschnitt wurde mit einer Rechentiefe von 250 Iterationen erzeugt. Seine Breite im Realteil beträgt nur 0,000484. Trotz dieser starken Vergrößerung treten erneut Miniaturen und verzweigte Randstrukturen auf. Man kann deshalb von angenäherter Selbstähnlichkeit sprechen; eine exakte Kopie der Gesamtfigur liegt jedoch nicht vor.

5.2 Nachwort

Sind Fraktale nur mathematische Spielerei, oder bewähren sie sich auch in der Praxis? Tatsächlich finden fraktale Verfahren in verschiedenen Bereichen Anwendung. Mit Algorithmen, die aus wenigen Befehlen bestehen, lassen sich künstliche Landschaften erzeugen, die von echten Landschaften kaum zu unterscheiden sind. Solche Verfahren können beispielsweise in der Filmproduktion eingesetzt werden, um aufwendige Modelle zu ergänzen oder zu ersetzen.

Eine weitere Anwendung liegt in der digitalen Bildübertragung und Bildkompression. Bildinformationen können durch geeignete mathematische Vorschriften kompakt beschrieben werden. Dadurch lassen sich Daten schneller übertragen, was insbesondere dort von Bedeutung ist, wo Speicherplatz und Übertragungszeit knapp oder teuer sind.

Die letzten Kapitel zeigen, dass die fraktale Geometrie ein umfangreiches Gebiet innerhalb der Mathematik darstellt. Im Rahmen einer Facharbeit kann nur ein allgemeiner Einblick gegeben werden. Die Grenzen des schulischen mathematischen Wissens sind schnell erreicht, da die Schulmathematik für viele Einzelheiten nicht ausreicht. Das Literaturverzeichnis nennt mehrere Werke, die sich mit besonderen fraktalen Strukturen und verwandten Themen befassen.

5.3 Literaturverzeichnis

[1] Benoît B. Mandelbrot. *The Fractal Geometry of Nature*. San Francisco: W. H. Freeman, 1982.

[2] Karl-Heinz Becker und Michael Dörfler. *Dynamische Systeme und Fraktale. Computergrafische Experimente mit Pascal*. 1. Aufl. Braunschweig: Vieweg, 1986.

[3] Heinz-Otto Peitgen und Dietmar Saupe, Hrsg. *The Science of Fractal Images*. New York: Springer, 1988.

[4] Heinz-Otto Peitgen und Peter H. Richter. *The Beauty of Fractals. Images of Complex Dynamical Systems*. Berlin, Heidelberg und New York: Springer, 1986.

[5] Hartmut Jürgens, Heinz-Otto Peitgen und Dietmar Saupe. *Chaos und Fraktale*. Heidelberg: Spektrum der Wissenschaft Verlagsgesellschaft, 1989.

[6] Herbert Pieper. *Die komplexen Zahlen. Theorie – Praxis – Geschichte*. Deutsch-Taschenbücher 44. Thun und Frankfurt am Main: Harri Deutsch, 1985.

[7] Detlef Laugwitz. *Ingenieurmathematik I*. BI-Hochschultaschenbücher 59. Mannheim: Bibliographisches Institut, 1964.

[8] Heinz König. *Analysis I*. Basel, Boston und Stuttgart: Birkhäuser, 1984.

[9] Immo Diener. *Computer-Kurzweil*. Heidelberg: Spektrum der Wissenschaft Verlagsgesellschaft, 1988.

Quellenhinweis Das Verzeichnis folgt Reihenfolge und Auswahl des Originalscans. Die Angaben wurden 2026 anhand von Bibliothekskatalogen und Verlagsangaben geprüft und um Untertitel und Reihen ergänzt. Berichtigt wurde Eintrag [3]: *The Science of Fractal Images* ist ein von Peitgen und Saupe herausgegebener Sammelband und erschien 1988, nicht 1986. Das Original nennt keine Seitenzahlen zu den einzelnen verwendeten Aussagen und Abbildungen.

5.4 Pascal-Listing

Der Ausdruck des Programms umfasst elf Blätter: Das erste beginnt mit `PROGRAM FRAKTAL;`, das letzte endet mit `END.`. Das folgende Listing ist eine zeichengetreue Abschrift dieses Ausdrucks aus dem Jahr 1990: Kommentare, Umlaute, Schreibweisen einschließlich Tippfehlern, Klammern und Einrückung sind wie gedruckt übernommen. Nur Zeilen, die der Drucker nach 80 Zeichen umbrochen hatte, wurden wieder zusammengefügt; das dabei am Papierrand verdeckte Zeichen wurde aus dem Zusammenhang ergänzt.

Eine vollständige Übersetzung ist dennoch nicht möglich, weil die eingebundenen Dateien `GEMCONST.PAS`, `TRIXCONST.PAS`, `GEMTYPE.PAS`, `TRIXTYPE.PAS`, `GEMSUBS` und `TRIXSUBS` nicht vorliegen. Auch Routinen wie `PAINT_RECT`, `GET_SCREEN` und `INIT_GEM` stammen daraus. Das Listing dokumentiert deshalb den zugänglichen Quelltext, ist aber ohne diese zeitgenössischen Bibliotheken kein eigenständig kompilierbares Programm.

Originaler Pascal-Quelltext von 1990

```
PROGRAM FRAKTAL;

 CONST

  {$I GEMCONST.PAS}         { Quelltext - Konstanten             }
  {$I TRIXCONST.PAS}        { Zusätzliche Quelltext - Konstanten }

 TYPE

  {$I GEMTYPE.PAS}          { Quelltext - Datentypen             }
  {$I TRIXTYPE.PAS}         { Zusätzliche Quelltext - Datentypen }

 VAR TASTE         : CHAR;           { einfacher Character                }
     I             : INTEGER;        { Wird im Zusammenhang mit Scan ver-
                                       wendet. Entspricht dem Zahlenwert
                                       der jeweiligen Taste                 }

     BILDSCHIRM    : SCREEN_TYPE;    { ST spezielles Bildschirm - Fomat    }
     BILD_SPEICHER : BOOLEAN;        { überprüft ob sich ein Bild im
                                       Speicher befindet. false entspricht
                                       einem leeren und true einem vollen
                                       Bildschirm-Speicher.                 }
     EN            : BOOLEAN;

  {$I GEMSUBS}              { Quelltext - Prozedur- und Funktionsköpfe }
  {$I TRIXSUBS}             { Weitere Prozedur- und Funktionsköpfe     }


 { Diese Routine fragt die Scancodes der Tastatur ab. Auf Grund dieser
   Routine ist es möglich auch spezielle Tasten (z.B. Funktionstasten)
   abzufragen. Dies ist mit dem herkömmlichen readln - Befehl nicht
   möglich.                                                          }

 FUNCTION BCONCIN(DEV : INTEGER) : LONG_INTEGER;BIOS(2);

 FUNCTION SCAN : INTEGER;

  BEGIN
   SCAN:=INT(BCONCIN (2) DIV (65535))
  END;

 { Prozedur zur Kursorsteuerung }

 PROCEDURE CURSOR(FUNKTION,RATE : INTEGER);XBIOS(21);

```

```
46
47  {-------------------------------------------------------------------------
48   FRACTALE AUSDRUCKEN
49   -------------------------------------------------------------------------}
50
51  PROCEDURE DRUCKEN;
52
53   CONST X_MAX = 639;
54         Y_MAX = 399;
55
56
57   VAR X,Y,I,J,CODE : INTEGER;
58       OK           : ARRAY[1..8] OF BOOLEAN;
59
60   BEGIN
61    IF (BILD_SPEICHER = FALSE) THEN
62     BEGIN
63      CUR_OFF;
64      CLEAR_SCREEN;
65      HIDE_MOUSE;
66      PAINT_STYLE(0);
67      PAINT_RECT(1,1,638,398);
68      PAINT_STYLE(1);
69      PAINT_RECT(4,4,632,392);
70      PAINT_STYLE(5);
71      PAINT_RECT(5,5,630,390);
72      PAINT_STYLE(1);
73      PAINT_RECT(232,189,194,34);
74      PAINT_STYLE(0);
75      PAINT_RECT(228,185,190,30);
76      GOTOXY(30,12);WRITELN('KEIN BILD IM SPEICHER');
77      I:=SCAN;
78     END
79    ELSE
80     BEGIN
81      PUT_SCREEN(BILDSCHIRM);
82      REWRITE(OUTPUT,'PRN:');    { Initialisierung des Druckers }
83      WRITELN(CHR(27),'U1');
84      WRITELN(CHR(27),'3',CHR(16));
85      X:=0;
86      Y:=0;
87      REPEAT
88       write(chr(27),'*',chr(6),chr((x_max+40+1) mod(256)),chr((x_max+40+1)div(256)));
89       for j:=1 to 40 do write(chr(0)); {linker Rand}
90       for x:=0 to x_max do
91        BEGIN
92         FOR I:=1 TO 8 DO OK[I]:=FALSE;
93         FOR I:=1 TO 8 DO
94          BEGIN
95           IF GET_PIXEL(X,Y+I-1)=BLACK THEN OK[I]:=TRUE;
96           IF I=8 THEN
97            BEGIN
98             CODE:=0;
99             IF OK[1]=TRUE THEN CODE:=CODE+128;
100            IF OK[2]=TRUE THEN CODE:=CODE+64;
101            IF OK[3]=TRUE THEN CODE:=CODE+32;
102            IF OK[4]=TRUE THEN CODE:=CODE+16;
103            IF OK[5]=TRUE THEN CODE:=CODE+8;
104            IF OK[6]=TRUE THEN CODE:=CODE+4;
105            IF OK[7]=TRUE THEN CODE:=CODE+2;
106            IF OK[8]=TRUE THEN CODE:=CODE+1;
107            WRITE(CHR(CODE));
108            IF X = X_MAX THEN FOR J:=1 TO (90*8-(X_MAX+1)-40) DO WRITE(CHR(0));
109           END;
110         END;
111       END;
112       WRITELN(CHR(27),'J',CHR(8));
```

```
      Y:=Y+8;
    UNTIL Y>Y_MAX+8;
    REWRITE(OUTPUT,'CON:');
   END;
 END;

{------------------------------------------------------------------------
 FRACTALE BERECHNEN
 ------------------------------------------------------------------------}

PROCEDURE BERECHNEN;

 VAR X_KOORDINATE,                  { x - Koordinate }
     Y_KOORDINATE,                  { y - Koordinate }
     RECHENTIEFE,                   { Rechentiefe    }
     ZAEHLER,                       { Zähler für Iterationen }
     X_Z,                           { Zähler für x - Koordinate }
     Y_Z           : INTEGER;       { Zähler für y - Koordinate }
     REAL_MIN,                      { Realteil - Minimum }
     REAL_MAX,                      { Realteil - Maximum }
     IMAG_MIN,                      { Imaginärteil - Minimum }
     IMAG_MAX,                      { Imaginärteil - Maximum }
     X_SCHRITT,                     { Schrittweite in x - Richtung }
     Y_SCHRITT,                     { schrittweite in y - Richtung }
     I_KONST,                       { Imaginärteil der Konstante }
     R_KONST,                       { Realteil der Konstante }
     REAL_ZAHL,                     { Neu errechneter Realteil }
     IMAG_ZAHL,                     { Neu errechneter Imaginärteil }
     REAL_Q,                        { Quadrierter Realteil }
     IMAG_Q       : REAL;           { Quadrierter Imaginäreil }
     STD,MIN,SEC  : INTEGER;        { Variabeln für Rechenzeit }


   {------------------------------------------------------------------------
    GESTALTUNG DES HINTERGRUNDS UND DER EINGABEMASKE
    ------------------------------------------------------------------------}

   PROCEDURE HINTERGRUND;

    VAR A : INTEGER;

    BEGIN
     CUR_OFF;
     CLEAR_SCREEN;
     HIDE_MOUSE;
     PAINT_STYLE(0);
     PAINT_RECT(1,1,638,398);
     PAINT_STYLE(1);
     PAINT_RECT(4,4,632,392);
     PAINT_STYLE(5);
     PAINT_RECT(5,5,630,390);

     { Gestaltung des Eingabefensters }

     PAINT_STYLE(0);
     PAINT_RECT(113,50,413,65);
     PAINT_STYLE(1);
     PAINT_RECT(116,53,407,59);
     PAINT_STYLE(5);
     PAINT_RECT(117,54,405,57);
     PAINT_STYLE(1);
     PAINT_RECT(125,60,389,45);
     PAINT_STYLE(0);
     PAINT_RECT(123,58,389,45);
     TEXT_STYLE(0);
     TEXT_HEIGTH(13);
     DRAW_STRING(235,85,'  Fractale berechnen  ');
```

```
180
181       { Gestaltung der Eingabebox }
182
183       PAINT_STYLE(0);
184       PAINT_RECT(113,120,413,246);
185       PAINT_STYLE(1);
186       PAINT_RECT(116,123,407,240);
187       PAINT_STYLE(5);
188       PAINT_RECT(117,124,405,238);
189       PAINT_STYLE(1);
190       PAINT_RECT(165,160,309,156);
191       PAINT_STYLE(0);
192       PAINT_RECT(163,158,309,156);
193       FOR A:=0 TO 3 DO
194        BEGIN
195         PAINT_STYLE(1);
196         PAINT_RECT(172,174+(A*32),143,25);
197         PAINT_STYLE(0);
198         PAINT_RECT(170,172+(A*32),143,25);
199         PAINT_STYLE(1);
200         PAINT_RECT(323,174+(A*32),140,25);
201         PAINT_STYLE(0);
202         PAINT_RECT(321,172+(A*32),140,25);
203        END;
204       GOTOXY(22,11);WRITELN('Realteil Minimum');
205       GOTOXY(22,13);WRITELN('Realteil Maximum');
206       GOTOXY(22,15);WRITELN('Imaginär Minimum');
207       GOTOXY(22,17);WRITELN('Imaginär Maximum');
208      END;
209
210
211  BEGIN
212
213   { Eingabe der Parameter und Fehlerüberprüfung }
214
215   REPEAT
216    HINTERGRUND;
217    CUR_ON;
218    CURSOR(3,0);
219    GOTOXY(42,11);READLN(REAL_MIN);
220    GOTOXY(42,13);READLN(REAL_MAX);
221    GOTOXY(42,15);READLN(IMAG_MIN);
222    GOTOXY(42,17);READLN(IMAG_MAX);
223    IF (REAL_MIN >= REAL_MAX) OR (IMAG_MIN >= IMAG_MAX) THEN
224     BEGIN
225      CUR_OFF;
226      PAINT_STYLE(1);
227      PAINT_RECT(165,160,309,156);
228      PAINT_STYLE(0);
229      PAINT_RECT(163,158,309,156);
230      DRAW_STRING(200,85,'E I N G A B E F E H L E R  !!!');
231      DRAW_STRING(210,200,'Achtung ! Das Minimum darf ');
232      DRAW_STRING(210,230,'nie größer als das Maximum ');
233      DRAW_STRING(210,260,'          sein !           ');
234      I:=SCAN;
235      CUR_ON;
236     END;
237   UNTIL (REAL_MAX > REAL_MIN) AND (IMAG_MAX > IMAG_MIN);
238
239   { Gestaltung des Hintergrunds }
240
241   CUR_OFF;
242   CLEAR_SCREEN;
243   HIDE_MOUSE;
244   PAINT_STYLE(0);
245   PAINT_RECT(1,1,638,398);
246   PAINT_STYLE(1);
```

```
247  PAINT_RECT(4,4,632,392);
248  PAINT_STYLE(5);
249  PAINT_RECT(5,5,630,390);
250
251  { Gestaltung der Eingabebox }
252
253  PAINT_STYLE(0);
254  PAINT_RECT(113,50,413,65);
255  PAINT_STYLE(1);
256  PAINT_RECT(116,53,407,59);
257  PAINT_STYLE(5);
258  PAINT_RECT(117,54,405,57);
259  PAINT_STYLE(1);
260  PAINT_RECT(125,60,389,45);
261  PAINT_STYLE(0);
262  PAINT_RECT(123,58,389,45);
263  TEXT_STYLE(0);
264  TEXT_HEIGTH(13);
265  DRAW_STRING(235,85,'  Fractale berechnen  ');
266  PAINT_STYLE(0);
267  PAINT_RECT(113,120,413,246);
268  PAINT_STYLE(1);
269  PAINT_RECT(116,123,407,240);
270  PAINT_STYLE(5);
271  PAINT_RECT(117,124,405,238);
272  PAINT_STYLE(1);
273  PAINT_RECT(165,160,309,156);
274  PAINT_STYLE(0);
275  PAINT_RECT(163,158,309,156);
276  CUR_ON;
277  GOTOXY(23,11);WRITELN('Bitte wählen Sie einen geeigneten');
278  GOTOXY(23,12);WRITELN('     Wert für die Anzahl der     ');
279  GOTOXY(23,13);WRITELN('          Iterationen !          ');
280  CUR_OFF;
281  TEXT_HEIGTH(4);
282  DRAW_STRING(175,230,'ACHTUNG !!!');
283  DRAW_STRING(175,238,'Wählen Sie für die Anzahl der Iterationen einen');
284  DRAW_STRING(175,246,'kleinen Wert, so ensteht ein grobes Bild ohne  ');
285  DRAW_STRING(175,254,'Feinstruktur. Bei gößeren Werten sind Fein-    ');
286  DRAW_STRING(175,262,'strukturen zu erkennen, die aber lange Rechen- ');
287  DRAW_STRING(175,270,'zeiten beanspruchen.(=> Werte zwischen 100 und ');
288  DRAW_STRING(175,278,'350 günstig )                                  ');
289
290  CUR_ON;
291  GOTOXY(37,18);WRITELN('>');
292  REPEAT
293   GOTOXY(38,18);READLN(RECHENTIEFE);
294  UNTIL RECHENTIEFE>0;
295  CUR_OFF;
296  HIDE_MOUSE;
297  CLEAR_SCREEN;
298
299  { Rechenzeit auf Null setzen }
300
301  STD:=0;MIN:=0;SEC:=0;
302  SETTIME(STD,MIN,SEC);
303
304  {------------------------------------------------------------------------
305   DER APFELMÄNNCHEN-ALGORITHMUS
306   ------------------------------------------------------------------------}
307
308  X_KOORDINATE:=400;
309  Y_KOORDINATE:=400;
310
311  { Berechnung der x- und y-Schrittweite }
312
313  X_SCHRITT:=ABS(((REAL_MAX)-(REAL_MIN))/X_KOORDINATE);
```

```
  Y_SCHRITT:=ABS(((IMAG_MAX)-(IMAG_MIN))/Y_KOORDINATE);

  I_KONST:=IMAG_Min+Y_SCHRITT;
  FOR Y_Z:=0 TO Y_KOORDINATE DO
   BEGIN
    R_KONST:=REAL_MIN+X_SCHRITT;
    FOR X_Z:=0 TO X_KOORDINATE DO
     BEGIN
      REAL_ZAHL:=R_KONST;
      IMAG_ZAHL:=I_KONST;
      ZAEHLER:=RECHENTIEFE;
      REPEAT
       REAL_Q:=REAL_ZAHL*REAL_ZAHL;
       IMAG_Q:=IMAG_ZAHL*IMAG_ZAHL;
       IMAG_ZAHL:=2*REAL_ZAHL*IMAG_ZAHL+I_KONST;
       REAL_ZAHL:=REAL_Q-IMAG_Q+R_KONST;
       ZAEHLER:=ZAEHLER-1;
      UNTIL (REAL_Q+IMAG_Q>4) OR (ZAEHLER=0);
      IF ODD(ZAEHLER) OR (ZAEHLER=0)THEN PUT_PIXEL(X_Z,Y_Z,1);
      R_KONST:=R_KONST+X_SCHRITT;
     END;
   I_KONST:=I_KONST+Y_SCHRITT;
  END;

  { Parameterbeschriftung }

  GOTOXY(55,3 );WRITELN('Realteil:');
  GOTOXY(55,4 );WRITELN('Minimum: ',REAL_MIN:8:8);
  GOTOXY(55,5 );WRITELN('Maximum: ',REAL_MAX:8:8);
  GOTOXY(55,7 );WRITELN('Imaginärteil:');
  GOTOXY(55,8 );WRITELN('Minimum: ',IMAG_MIN:8:8);
  GOTOXY(55,9 );WRITELN('Maximum: ',IMAG_MAX:8:8);
  GOTOXY(55,11);WRITELN('Rechentiefe: ',RECHENTIEFE);
  GETTIME(STD,MIN,SEC);
  GOTOXY(55,13);WRITELN('Rechenzeit:  ',STD,':',MIN,':',SEC);
  GET_SCREEN(BILDSCHIRM); { Aktueller Bildschirminhalt wird im
                            Bildschirmspeicher abgelegt }
  BILD_SPEICHER:=TRUE;
  I:=SCAN;
  SHOW_MOUSE;
END;

{------------------------------------------------------------------------
 AUSGABE DER AKTUELLEN PARAMETER
 ------------------------------------------------------------------------}

PROCEDURE PARAMETER;

 BEGIN
  IF (BILD_SPEICHER = FALSE) THEN
   BEGIN
    CUR_OFF;
    CLEAR_SCREEN;
    HIDE_MOUSE;
    PAINT_STYLE(0);
    PAINT_RECT(1,1,638,398);
    PAINT_STYLE(1);
    PAINT_RECT(4,4,632,392);
    PAINT_STYLE(5);
    PAINT_RECT(5,5,630,390);
    PAINT_STYLE(1);
    PAINT_RECT(232,189,194,34);
    PAINT_STYLE(0);
    PAINT_RECT(228,185,190,30);
    GOTOXY(30,12);WRITELN('KEIN BILD IM SPEICHER');
   END
  ELSE
```

```
   PUT_SCREEN(BILDSCHIRM);
   I:=SCAN;
 END;

{-------------------------------------------------------------------------
 FRACTALE SPEICHERN UND LADEN
 -------------------------------------------------------------------------}

PROCEDURE IM_EXPORT;

 VAR FEHLER     : ERR_CODE;      { Fehler-Code beim Laden/Speichern }
     PFAD       : STRING;        { Pfad-Name der Datei              }
     NAME       : STRING;        { Name der Datei                   }
     D          : INTEGER;

BEGIN

 REPEAT

  { Gestaltung des Hintergrunds }

  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);

  { Gestaltung der Eingabebox }

  PAINT_STYLE(0);
  PAINT_RECT(213,100,213,65);
  PAINT_STYLE(1);
  PAINT_RECT(216,103,207,59);
  PAINT_STYLE(5);
  PAINT_RECT(217,104,205,57);
  PAINT_STYLE(1);
  PAINT_RECT(225,110,189,45);
  PAINT_STYLE(0);
  PAINT_RECT(223,108,189,45);
  TEXT_STYLE(0);
  TEXT_HEIGTH(13);
  DRAW_STRING(235,125,'  Erstellte Bilder  ');
  DRAW_STRING(235,144,'Im- bzw. Exportieren');

  { Unterer Kasten in der Mitte }

  PAINT_STYLE(0);
  PAINT_RECT(213,170,213,150);
  PAINT_STYLE(1);
  PAINT_RECT(216,173,207,144);
  PAINT_STYLE(5);
  PAINT_RECT(217,174,205,142);
  FOR D:=0 TO 2 DO BEGIN
   PAINT_STYLE(0);
   PAINT_RECT(220,185+(D*42),40,39);
   PAINT_STYLE(1);
   PAINT_RECT(225,190+(D*42),32,31);
   PAINT_STYLE(0);
   PAINT_RECT(223,188+(D*42),32,31);
   PAINT_RECT(265,185+(D*42),154,39);
   PAINT_STYLE(1);
   PAINT_RECT(270,190+(D*42),146,31);
   PAINT_STYLE(0);
   PAINT_RECT(268,188+(D*42),146,31);
  END;
```

```
448
449   { Beschriftung der Menüboxen }
450
451   TEXT_STYLE(0);
452   TEXT_HEIGTH(13);
453   DRAW_STRING(227,208,'F 6');
454   DRAW_STRING(227,250,'F 7');
455   DRAW_STRING(227,292,'ESC');
456   TEXT_HEIGTH(4);
457   DRAW_STRING(278,200,'   Erstellte Bilder   ');
458   DRAW_STRING(278,209,'importieren bzw. laden');
459   DRAW_STRING(278,238,'   Erstellte Bilder   ');
460   DRAW_STRING(278,246,'   exportieren bzw.   ');
461   DRAW_STRING(278,254,'      speichern       ');
462   DRAW_STRING(278,288,' Zurück ins Hauptmenü ');
463   I:=SCAN;
464   IF I=64 THEN
465    BEGIN
466     SHOW_MOUSE;
467     NAME:='';
468     PFAD:='A:\*.*';
469     IF GET_IN_FILE(PFAD,NAME) THEN
470      BEGIN
471       CLEAR_SCREEN;
472       FEHLER:=READ_SCREEN(1,NAME,BILDSCHIRM);
473       PUT_SCREEN(BILDSCHIRM);
474       BILD_SPEICHER:=TRUE;
475       I:=SCAN;
476      END;
477    END;
478   IF I=65 THEN
479    BEGIN
480     IF BILD_SPEICHER = FALSE THEN
481      BEGIN
482       CUR_OFF;
483       CLEAR_SCREEN;
484       HIDE_MOUSE;
485       PAINT_STYLE(0);
486       PAINT_RECT(1,1,638,398);
487       PAINT_STYLE(1);
488       PAINT_RECT(4,4,632,392);
489       PAINT_STYLE(5);
490       PAINT_RECT(5,5,630,390);
491       PAINT_STYLE(1);
492       PAINT_RECT(232,189,194,34);
493       PAINT_STYLE(0);
494       PAINT_RECT(228,185,190,30);
495       GOTOXY(30,12);WRITELN('KEIN BILD IM SPEICHER');
496       I:=SCAN;
497      END
498     ELSE
499      BEGIN
500       SHOW_MOUSE;
501       CLEAR_SCREEN;
502       PARAMETER;
503       NAME:='';
504       PFAD:='A:\*.*';
505       IF GET_IN_FILE(PFAD,NAME) THEN
506        BEGIN
507         FEHLER:=WRITE_SCREEN(1,NAME,BILDSCHIRM);
508        END;
509      END;
510    END;
511  UNTIL I=1;
512 END;
513
514 {------------------------------------------------------------------------
```

```
 INFORMATION
 ------------------------------------------------------------------------}

PROCEDURE INFO;

 BEGIN
  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);
  PAINT_STYLE(0);
  PAINT_RECT(50,40,540,320);
  PAINT_STYLE(1);
  PAINT_RECT(60,50,520,300);
  PAINT_STYLE(0);
  PAINT_RECT(58,48,520,300);
  TEXT_HEIGTH(13);
  TEXT_STYLE(0);
  DRAW_STRING(70,70,'Information');
  DRAW_STRING(70,110,'Das vorliegende Computerprogramm ermöglicht die graphische');
  DRAW_STRING(70,130,'Darstellung der Mandelbrot-Menge. Hierzu müssen in F2 fünf');
  DRAW_STRING(70,150,'Eingabegrößen festgelegt werden. Bitte beachten Sie bei der');
  DRAW_STRING(70,170,'Eingabe,daß Sie keine Buchstaben eingeben. Verwenden Sie ');
  DRAW_STRING(70,190,'für Kommazahlen einen Punkt. Bei einer falschen Eingabe kann');
  DRAW_STRING(70,210,'durchaus vorkommen,daß das Programm abstürzt. Dieses schwer-');
  DRAW_STRING(70,230,'wiegende Problem konnte nicht gelöst werden, da es sich um');
  DRAW_STRING(70,250,'eine aufwendige Routine handelt ( Die Eingabe müßte als String');
  DRAW_STRING(70,270,'deklariert werden und auf jede Stelle untersucht werden ).');
  DRAW_STRING(70,290,'Dieses Programm wurde in Pascal erstellt. Pascal ist im Gegen-');
  DRAW_STRING(70,310,'satz zu Assembler relativ langsam. Dennoch wünsche ich dem ');
  DRAW_STRING(70,330,'Benutzer beim Erstellen von Fractale viel Spaß. (TASTE !)');
  I:=SCAN;
 END;

{------------------------------------------------------------------------
 HAUPTMENÜ
 ------------------------------------------------------------------------}
PROCEDURE HAUPTMENU;

 VAR A : INTEGER;

 BEGIN

  { Gestaltung des Hintegrunds }

  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);

  { Fenster in der Mitte }

  PAINT_STYLE(0);
  PAINT_RECT(182,20,280,360);
  PAINT_RECT(184,22,276,356);
  PAINT_STYLE(4);
  PAINT_RECT(185,23,274,354);

  { Hintergrund der Menübox }

  PAINT_STYLE(0);
```

```
  PAINT_RECT(190,28,264,344);
  PAINT_RECT(192,30,260,340);
  PAINT_RECT(193,31,258,338);
  PAINT_STYLE(5);
  PAINT_RECT(194,32,256,336);

  { Beschriftung der Menübox }

  PAINT_STYLE(0);
  PAINT_RECT(199,40,245,55);
  PAINT_RECT(201,42,241,51);
  TEXT_STYLE(1);
  TEXT_HEIGTH(13);
  DRAW_STRING(250,65,'MANDELBROT - MENGEN');
  TEXT_HEIGTH(4);
  DRAW_STRING(220,78,'Das Computerprogramm zur Facharbeit');
  DRAW_STRING(220,88,'         von Urban Bettag          ');
  FOR A:=1 TO 6 DO
   BEGIN
    PAINT_STYLE(0);
    PAINT_RECT(199,79+(A*32),50,29);PAINT_RECT(279,79+(A*32),146,29);
    PAINT_STYLE(1);
    PAINT_RECT(204,84+(A*32),42,21);PAINT_RECT(284,84+(A*32),138,21);
    PAINT_STYLE(0);
    PAINT_RECT(202,82+(A*32),42,21);PAINT_RECT(282,82+(A*32),138,21);
   END;
  TEXT_STYLE(0);
  TEXT_HEIGTH(13);
  DRAW_STRING(212,130,'F 1');TEXT_HEIGTH(4);DRAW_STRING(288,122,'   Erstelle Bilder  ');
                                            DRAW_STRING(292,130,'Im- bzw. Exportieren');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,162,'F 2');TEXT_HEIGTH(4);DRAW_STRING(292,158,' Fractale berechnen ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,194,'F 3');TEXT_HEIGTH(4);DRAW_STRING(292,186,' Aktuelle Parameter ');
                                            DRAW_STRING(292,193,'     einblenden     ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,226,'F 4');TEXT_HEIGTH(4);DRAW_STRING(292,222,' Fractal ausdrucken ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,258,'F 5');TEXT_HEIGTH(4);DRAW_STRING(274,294,'  Information für   ');
                                            DRAW_STRING(292,258,'     Benutzer       ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,290,'F10');TEXT_HEIGTH(4);DRAW_STRING(292,286,'  Programm beenden  ');
  TEXT_HEIGTH(13);
 END;

{-------------------------------------------------------------------------
 BEENDEN
 -------------------------------------------------------------------------}

 PROCEDURE BEENDEN;

 BEGIN
  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);
  PAINT_STYLE(1);
  PAINT_RECT(210,180,220,40);
  PAINT_STYLE(0);
  PAINT_RECT(208,178,220,40);
  DRAW_STRING(220,200,'Sind Sie sicher (J/N) ?');
  I:=SCAN;
  IF I=36 THEN EN:=FALSE;
  IF I=49 THEN EN:=TRUE;
```

```
 END;

{------------------------------------------------------------------------
 HAUPTPROGRAMM
 ------------------------------------------------------------------------}

BEGIN

 { GEM - Routinen werden aufgerufen und instaliert

   Init_Gem
   --------

   Die Funktion führt alle notwendigen Operationen durch, um GEM mitzuteilen
   daß nun eine Applikation an den Segnungen des Betriebssystems teilhaben
   möchte. Tritt ein Fehler auf so liefert die Funktion Init_gem der Wert -1
   Eine erfolgreiche Operation liefert immer einen positiven Wert.

   Exit_Gem
   --------

   Beendet ein GEM - Programm (Graphic Environment Manager). Die Prozedur
   teil GEM mit, daß ab nun keine weiteren GEM-Aufrufe getätigt werden.
   Bei einem fehlerhaften Aufruf von Init_Gem sollte man diese Prozedur nicht
   aufrufen. }

 IF INIT_GEM >=0 THEN
  BEGIN
   BILD_SPEICHER:=FALSE;
   EN:=TRUE;
   REPEAT
    HAUPTMENU;
    I:=SCAN;
    IF I=59 THEN IM_EXPORT;
    IF I=60 THEN BERECHNEN;
    IF I=61 THEN PARAMETER;
    IF I=62 THEN DRUCKEN;
    IF I=63 THEN INFO;
    IF I=68 THEN BEENDEN;
    SHOW_MOUSE;
   UNTIL EN=FALSE;
   EXIT_GEM;
  END;
END.
```

Prüfvermerk zum Programm

- **Historischer Befund:** Das Programm ist erkennbar für einen Atari-ST-Rechner mit GEM-Oberfläche und BIOS-/XBIOS-Aufrufen geschrieben. Bezeichner wie `TEXT_HEIGTH` wurden nicht modernisiert, weil sie aus den fehlenden Bibliotheken stammen können.
- **Scancode:** Der Rückgabewert von `BCONCIN` enthält den Scancode im oberen Wort. Dafür wäre eine Division durch 65536 = 2^{16} beziehungsweise eine Verschiebung um 16 Bit zu erwarten. Im Ausdruck steht `DIV (65535)`; dies ist sehr wahrscheinlich ein Fehler im Originalprogramm und wurde bewusst beibehalten.
- **Menübeschriftung:** Im Hauptmenü steht die Beschriftung „Information für“ bei $y = 294$, die zugehörige zweite Zeile „Benutzer“ bei $y = 258$. Die beiden Wörter erscheinen damit getrennt auf dem Bildschirm; auch dies ist ein Fehler des Originals und bleibt erhalten.
- **Rastergrenzen:** Die Schleifen laufen jeweils von 0 bis einschließlich 400, also über 401

Werte. Zugleich werden Real- und Imaginärteil schon vor dem ersten Bildpunkt um eine Schrittweite erhöht. Dadurch werden die angegebenen Minima ausgelassen und am Ende Werte oberhalb der Maxima erreicht. Außerdem kann die y-Koordinate 400 bei einem Bildschirmbereich von 0 bis 399 außerhalb des vorgesehenen Rasters liegen.

- **Iteration:** Für jeden Parameter c beginnt die Rechnung im Programm mit $z = c$, nicht mit $z_0 = 0$. Für den reinen Entweichungstest entspricht dies einer um einen Schritt verschobenen Zählung; für die verwendete Schwarz-Weiß-Färbung verändert es jedoch die Parität der Iterationszahl. Zudem wird die Abbruchsumme `REAL_Q+IMAG_Q` aus den Quadraten vor der gerade berechneten Aktualisierung gebildet.
- **Darstellung:** Außenpunkte werden nach der Parität des noch vorhandenen Zählers schwarz oder weiß gezeichnet. Die Streifen zeigen daher Fluchtzeitklassen. Sie sind kein eigenständiger Beweis für die Zugehörigkeit eines Punktes zur Mandelbrot-Menge.
- **Heutige Einordnung:** Mit den fehlenden GEM-/TRIX-Dateien und einem passenden historischen Pascal-System ließe sich die Syntax genauer prüfen. Ohne diese Bestandteile wäre jede angeblich lauffähige Rekonstruktion spekulativ; deshalb wurde hier nur der tatsächlich zugängliche Programmtext aufgenommen.

5.5 Python-Nachbearbeitung von 2026

Die folgenden Python-Programme gehören nicht zur ursprünglichen Facharbeit von 1990. Sie wurden im Jahr 2026 für die digitale Nachbearbeitung erstellt. Das erste Programm berechnet die neuen Schwarz-Weiß-, Farb- und Detaildarstellungen der Mandelbrot-Menge. Das zweite Programm erzeugt die dreidimensionale Fluchtzeitdarstellung. Beide Programme verwenden die mathematische Iteration $z_{n+1} = z_n^2 + c$ mit dem Startwert $z_0 = 0$.

Abgedruckt ist jeweils nur der Rechenkern; die Zeilennummern entsprechen der vollständigen Datei. Die übrigen Programmteile erzeugen lediglich Beschriftung und Bildaufbau. Die vollständigen Skripte `generate_fractals.py`, `generate_3d_fractal.py`, `generate_diagrams.py` und `generate_ifs.py` sind nicht abgedruckt.

Fluchtzeiten für die zweidimensionalen Abbildungen

`generate_fractals.py`: Funktion `escape_times`

```
40 def escape_times(bounds, iterations, width=1000):
41     """Berechne die erste Fluchtiteration fuer z_0=0, z_(n+1)=z_n^2+c."""
42     xmin, xmax, ymin, ymax = bounds
43     height = max(2, round(width * (ymax - ymin) / (xmax - xmin)))
44     x = np.linspace(xmin, xmax, width, dtype=np.float64)
45     y = np.linspace(ymin, ymax, height, dtype=np.float64)
46     c = x[np.newaxis, :] + 1j * y[:, np.newaxis]
47     z = np.zeros_like(c)
48     active = np.ones(c.shape, dtype=bool)
49     escaped = np.full(c.shape, iterations, dtype=np.uint16)
50 
51     for n in range(1, iterations + 1):
52         z[active] = z[active] * z[active] + c[active]
53         newly = active & (z.real * z.real + z.imag * z.imag > 4.0)
54         escaped[newly] = n
55         active[newly] = False
56         if not active.any():
57             break
58     return escaped
```

Für jeden Bildpunkt c wird die Folge ab $z_0 = 0$ iteriert, bis $|z_n|^2 > 4$ gilt oder das Iterationslimit erreicht ist. Gespeichert wird der Index der ersten Überschreitung, also die Fluchtzeit.

Höhenfeld für die dreidimensionale Darstellung

`generate_3d_fractal.py`: Funktion `height_field`

```
44 def height_field(nx=230, ny=190):
45     xmin, xmax, ymin, ymax = BOUNDS
46     xs = np.linspace(xmin, xmax, nx)
47     ys = np.linspace(ymin, ymax, ny)
48     c = xs[np.newaxis, :] + 1j * ys[:, np.newaxis]
49     z = np.zeros_like(c)
50     active = np.ones(c.shape, dtype=bool)
51     smooth = np.full(c.shape, float(MAX_ITER), dtype=np.float64)
52 
53     for iteration in range(1, MAX_ITER + 1):
54         z[active] = z[active] * z[active] + c[active]
55         radius2 = z.real * z.real + z.imag * z.imag
56         escaped = active & (radius2 > 4.0)
```

```
        if escaped.any():
            modulus = np.sqrt(radius2[escaped])
            smooth[escaped] = (
                iteration + 1.0 - np.log(np.log(modulus)) / np.log(2.0)
            )
        active[escaped] = False
        if not active.any():
            break

    # Schnelles Entweichen ergibt grosse Hoehen. Nicht entwichene Punkte sind
    # der ebene See auf Hoehe null. Die Wurzel macht die Kueste besser sichtbar.
    height = np.zeros_like(smooth)
    outside = ~active
    speed = np.clip((MAX_ITER - smooth[outside]) / MAX_ITER, 0.0, 1.0)
    height[outside] = 0.20 + 1.85 * np.sqrt(speed)
    return xs, ys, height, active
```

Die Fluchtzeit wird hier geglättet und in eine Höhe umgerechnet: Je schneller ein Punkt entweicht, desto höher liegt er. Nicht entwichene Punkte bilden die ebene Fläche des „Mandelbrot-Sees“.

Änderungsnachweis

Die Tabelle nennt die wichtigsten inhaltlichen Korrekturen der Originalseiten 20 bis 24. Sie ist nicht vollständig.

Seite	Ursprüngliches Problem	Korrektur	Wirkung
20	Startwert und Index verschoben	$z_0 = 0$, $z_1 = k$, $z_2 = k^2 + k$	Rekursion und Programmzählung stimmen überein
21	Parameter k fällt weg	Fallunterscheidung ausdrücklich auf $k = 0$ begrenzt	Der vorhandene Abschnitt bleibt als Sonderfall erhalten
22	$\mathcal{M}$ über Konvergenz gegen null definiert	Beschränktheit der kritischen Bahn	Korrekte Mengendefinition
22–23	Endlicher Test als Zugehörigkeitsbeweis	Nichtentweichen bis N als Näherung	Algorithmische Aussage wird präzise
23	Neue Komponenten nicht zurückgeschrieben	a und b nach jedem Schritt aktualisieren	Pseudocode führt tatsächlich eine Iteration aus
24	Parameter- und Startwertebene vermischt	Mandelbrot- und Julia-Ebene trennen	Zusammenhang beider Mengen wird korrekt beschrieben

Seitenangaben beziehen sich auf das Original von 1990.

Quellenbasis

Grundlage sind die Originalscans `IMG_1917` bis `IMG_1960` mit den Druckseiten 1 bis 44. Die Dateinummer `IMG_1950` fehlt; die zugehörige Stelle wird nach Angabe des Verfassers durch die später fotografierte Farbtafel `IMG_1960` geschlossen. Für die Teilprüfung des Programms wurde außerdem der sichtbare Programmkern in `IMG_1965` herangezogen. Die historischen Korrekturen verwenden mathematische Grundlagen, die bereits 1990 verfügbar waren; spätere Erkenntnisse sind ausdrücklich als solche gekennzeichnet.